

RiseUp — Stage 1 Report

Design & Architecture

Intelligent Alarm & Task Reminder System powered by an AI Agent

Stage 1 deliverable — complete design documentation, UML diagrams and design analysis

About This Report

This report combines **all Stage 1 design documents** and **all UML diagrams** of the RiseUp project into one document, and adds a summary, diagram walkthroughs and cross-reference tables. Chapters 1–6 reproduce the original files in `docs/stage-1-design/` word for word (the originals are unchanged), except that links between files now point to the matching chapter in this report.

Report Structure

Chapter	Source	Contents
Front matter	added	Executive Summary , Introduction , System Overview
1. Project Scope & Description	<code>project-scope.md</code>	Problem, target users, why an AI agent, AI models, architecture, F01–F14 summary, scope and success criteria.
2. Feature Specification & CLI Design	<code>feature-specification.md</code>	Canonical 14 features (F01–F14), each with the 8 required fields, plus the CLI design.
3. Use Cases	<code>use-cases.md</code>	UC-01 – UC-09 with actors, flows, alternatives and related features.
4. Design Decisions & Justifications	<code>design-decisions.md</code>	Architecture, AI integration, data/persistence, testing/reliability and UX decisions with alternatives.
5. Design Patterns	<code>design-patterns.md</code>	Strategy, Adapter, Command, Observer, Factory Method — problem, classes, roles, rationale.
6. Feature-to-Design Traceability	<code>feature-to-design-mapping.md</code>	Traceability table mapping every feature to use cases, classes, methods, sequence diagrams and patterns.
7. UML Diagrams	<code>uml-diagrams/*.png</code>	10 diagrams (architecture, class, use-case, SD-01 – SD-07), each with a detailed walkthrough
8. Consolidated Design Analysis	added	Traceability matrix, coverage, class catalogue, decision index, fallbacks, risks, test strategy, success criteria
9. Glossary	added	Terms, technologies and identifiers
10. Conclusion and Next Steps	added	Stage 1 summary; Stage 2 and 3 plan

Stage 1 Deliverables Checklist

Deliverable	Where in this report
Project scope & problem statement	Chapter 1
Major features definition (F01–F14, 8 fields each)	Chapter 2
CLI design	Chapter 2, Section 3
Use cases (UC-01 – UC-09)	Chapter 3
Design decisions & justifications	Chapter 4
Design patterns (5)	Chapter 5
Feature-to-design mapping / traceability	Chapter 6, Chapter 8
UML diagrams (class, use-case, sequence, architecture)	Chapter 7

Table of Contents

- [Executive Summary](#)

- [Introduction](#)
- [System Overview](#)

- 1. [Project Scope & Description](#)
- 2. [Feature Specification & CLI Design](#)
- 3. [Use Cases](#)
- 4. [Design Decisions & Justifications](#)
- 5. [Design Patterns](#)
- 6. [Feature-to-Design Traceability](#)
- 7. [UML Diagrams](#)
 - 1. [Architecture Diagram](#)
 - 2. [Class Diagram](#)
 - 3. [Use Case Diagram](#)
 - 4. [SD-01 — Sequence Diagram — Alarm / Verify / Check-in](#)
 - 5. [SD-02 — Sequence Diagram — Agent Escalation Loop](#)
 - 6. [SD-03 — Sequence Diagram — Reminders & Prioritization](#)
 - 7. [SD-04 — Sequence Diagram — Behavioral Learning](#)
 - 8. [SD-05 — Sequence Diagram — CLI / Natural-Language Remind](#)
 - 9. [SD-06 — Sequence Diagram — AI Recommendations](#)
 - 10. [SD-07 — Sequence Diagram — History & Decision Explanation](#)
- 8. [Consolidated Design Analysis](#)
 - [8.1 Full traceability matrix](#)
 - [8.2 Use-case coverage](#)
 - [8.3 Pattern coverage](#)
 - [8.4 Class catalogue](#)
 - [8.5 Design decision index](#)
 - [8.6 Error-handling and fallback summary](#)
 - [8.7 Risk register](#)
 - [8.8 Test strategy overview](#)
 - [8.9 Success-criteria checklist](#)
- 9. [Glossary](#)
- 10. [Conclusion and Next Steps](#)

Executive Summary

RiseUp is an intelligent alarm and task-reminder system for people who struggle to wake up and to act on reminders. Conventional alarms can be dismissed half-asleep and static reminders are easy to ignore; neither adapts to what actually works for an individual. RiseUp addresses this with three ideas:

1. **Accountable wake-up.** An alarm cannot be dismissed until the user completes a *verification challenge* (photo object detection, math puzzle, trivia or a custom task). A *post-wake check-in* 5–10 minutes later confirms the user is still awake.
2. **Agent-driven escalation.** If the alarm or check-in is ignored, an AI agent (`AgentController`) observes the user's history, decides on the best next channel (phone call, SMS, Alexa alert or a consenting emergency contact), acts through tool adapters, observes the outcome and re-decides if the channel failed. A deterministic *fallback ladder* guarantees escalation even when the LLM is unavailable.
3. **Learning and explainability.** A nightly learning job aggregates logs into a `UserProfile` (success rate per channel, wake-time patterns, completion rate per category). The agent uses this memory for future decisions, offers personalised recommendations, and stores the reasoning and confidence of every AI decision so the user can always ask "Why?".

The system has two interfaces over **one FastAPI backend**: a **React + TypeScript web GUI** and a **riseup CLI**. AI is provided by **Anthropic Claude (`claude-sonnet-4-5`)** and **OpenAI (`gpt-4o`)** behind a common `LLMProvider` interface. Data is stored in **PostgreSQL** with **Redis** for caching, real-time messaging and background jobs.

Stage 1 at a glance

Item	Count / Result
Features specified (8 fields each)	14 (F01 – F14) — 9 deterministic, 2 hybrid (F07, F14), 3 AI with validation/fall-back (F04, F09, F13)
Use cases	9 (UC-01 – UC-09)
Design decisions documented with alternatives	16 (AD-01–05, AI-01–03, DP-01–03, TR-01–02, UX-01–03)
Design patterns (with problem, classes, roles, rationale)	5 — Strategy, Adapter, Command, Observer, Factory Method
UML diagrams	10 — architecture, class, use-case, 7 sequence diagrams (SD-01 – SD-07)
Interfaces	GUI (React) + CLI (<code>riseup</code>) over the same REST API
AI providers	Claude <code>claude-sonnet-4-5</code> , OpenAI <code>gpt-4o</code>
Planned agent behavioural tests	BR-01 – BR-03 (KUMA) plus deterministic unit tests (pytest)

Introduction

Purpose of this report

This report is the **Stage 1 (Design & Architecture) deliverable** for the RiseUp project. It brings every Stage 1 artifact together in one document so that a reader or grader can review the complete design, from the problem statement down to individual method calls, without moving between files.

How this report is organised

Part	Contents
Front matter	Executive summary, introduction, system overview (this part)
Chapters 1 – 6	The six Stage 1 design documents, reproduced in full. Each chapter opens with a short <i>Chapter overview</i> box written for this report
Chapter 7	All UML diagrams, each with a detailed walkthrough of participants, message flow and alternative paths
Chapter 8	Consolidated analysis: traceability matrices, class catalogue, decision index, error/fallback summary, risk register, test strategy
Chapter 9	Glossary of terms, abbreviations and identifiers
Chapter 10	Conclusion and next steps (Stages 2 and 3)

The original files in `docs/stage-1-design/` are the source of truth. Chapters 1–6 reproduce them word for word, except that links between files now point to the matching chapter in this report. The front matter, the chapter overview boxes, the diagram walkthroughs and Chapters 8–10 were added for this report and are built only from information already in those documents and diagrams.

Identifier conventions

Prefix	Meaning	Defined in
F01 – F14	Feature (canonical list)	Chapter 2 – Feature Specification
UC-01 – UC-09	Use case	Chapter 3 – Use Cases
SD-01 – SD-07	Sequence diagram	Chapter 6 – Traceability , Chapter 7
AD-xx	Architectural decision	Chapter 4 – Design Decisions
AI-xx	AI integration decision	Chapter 4
DP-xx	Data & persistence decision	Chapter 4
TR-xx	Testing & reliability decision	Chapter 4
UX-xx	User-experience decision	Chapter 4
BR-xx	Agent behavioural test (KUMA)	Chapter 4 , TR-01
(new)	Class introduced by the feature specification	Chapter 2

Feature type legend

Type	Meaning
Deterministic	Plain code with no LLM involvement; fully unit-testable
AI	The LLM produces the core output; deterministic code validates it before use
Hybrid	The LLM decides or extracts, and deterministic code validates and executes (AD-02 / AI-02). Every hybrid/AI feature has a documented fallback

System Overview

Technology stack

Layer	Technology	Role
Web client	React + TypeScript, Vite	Dashboard for alarms, tasks, insights, history, preferences
CLI client	Python, Typer (or argparse), httpx	riseup commands over the REST API
Backend	FastAPI (Python 3.10+)	REST + WebSocket API, controllers, services
Scheduling	APScheduler	Alarm and reminder trigger jobs
Background jobs	Celery (on Redis)	Escalation tasks, nightly learning
Persistence	PostgreSQL	Users, alarms, reminders, check-ins, logs, analytics
Cache / real-time	Redis	Profile & stats cache, sessions, pub/sub for WebSocket
AI	Anthropic claude-sonnet-4-5 , OpenAI gpt-4o	Escalation, prioritisation, recommendations, NL parsing
Telephony	Twilio	Phone calls and SMS
Smart home	Amazon Alexa	Alexa alerts / notifications
Push	Firebase Cloud Messaging	Check-in and reminder notifications
Vision	Camera + computer-vision library	Photo object-detection verification
Testing	pytest; KUMA	Deterministic unit tests; agent behavioural tests

Actors

Actor	Kind	Participates in
User	Primary (human)	UC-01, UC-02, UC-04, UC-06, UC-07, UC-08, UC-09
System / Scheduler	Primary (time-driven)	UC-02, UC-03, UC-05
LLM Service (Claude / OpenAI)	Secondary (external)	UC-03, UC-04, UC-05, UC-07
Twilio	Secondary (external)	UC-03, UC-08
Alexa	Secondary (external)	UC-03
Camera / Vision	Secondary (device)	UC-02
Firebase (push)	Secondary (external)	UC-02, UC-05
Emergency contact	Secondary (human, consent required)	UC-03 (last resort), UC-08

Where the AI is used – and where it is not

AI is used for	Deterministic code is used for
Choosing the escalation channel (F04)	Scheduling alarms and reminders (APScheduler)

AI is used for	Deterministic code is used for
Ranking simultaneous reminders (F09)	Validating math / trivia answers
Parsing natural-language reminders (F07)	Photo verification (vision library)
Generating personalised recommendations (F13)	Executing calls / SMS / push / Alexa (tool adapters)
Producing reasoning text for explanations (F14)	Computing statistics and learning metrics (F11)
	Storing and querying data (PostgreSQL)
	Fallback ladder when the LLM fails (UX-01)

The agent loop

The agent is a multi-step loop, not a single "prompt → text" call (AD-05):

1. **Observe:** load short-term context (the current alarm or reminder and the last tool result) and long-term memory (`UserProfile` aggregates and `EscalationLog` history).
2. **Decide:** `PromptBuilder` builds a structured prompt. `LLMClient` sends it to the configured `LLMProvider`. `ResponseParser` validates the JSON action {method, reasoning, confidence, fallback}.
3. **Act:** `ToolManager` runs the action through a `Tool` adapter (`TwilioClient`, `AlexaClient`, `FirebaseClient`).
4. **Observe outcome:** record success or failure in `EscalationLog` and update `UserProfile` learning data.
5. **Re-decide or stop:** if the tool failed and alternatives remain, decide again. Otherwise stop, falling back to the deterministic ladder if needed.

The loop is used by F04, F05, F07 (side effects after confirmation) and F13.

Major non-functional qualities

Quality	How the design achieves it
Reliability	AI never executes; every external call has a fallback (TR-02); fallback ladder (UX-01); ACID storage (DP-01)
Explainability	Each AI decision stores reasoning, confidence, inputs and a fallback flag (F14)
Privacy & consent	Minimal personal data; emergency contact only after explicit consent (F06); quiet hours respected
Extensibility	Strategy / Adapter / Factory Method / Command let new tasks, providers, tools and CLI verbs be added without editing callers
Testability	Deterministic components are unit-tested; adapters are mocked; AI behaviour is checked with KUMA behavioural tests
Performance	Pre-aggregated <code>UserProfile</code> metrics (DP-03); Redis caching (DP-02); cached analytics results
Portability	Web GUI + CLI on Windows, macOS and Linux

RiseUp - Project Scope & Description

Chapter 1 — source: docs/stage-1-design/project-scope.md

Chapter overview. Chapter 1 defines *what* RiseUp is and *why* it needs an AI agent. It covers the problem statement and target users, the solution (GUI, CLI, agent, memory, tools), what the agent can do, the AI models and how they connect to the rest of the system, assumptions and constraints, the F01–F14 feature summary, the overall architecture, items out of scope, and the success criteria Stage 1 is measured against.

Key takeaways: waking and reminding are multi-step decisions under uncertainty, so an agent fits; the deterministic parts never depend on the LLM succeeding; consent and explainability are hard constraints.

Canonical feature IDs are **F01–F14** in [feature-specification.md](#). This document answers Stage 1 Task 1.1 (problem, users, agent, models, architecture).

1. Problem Statement

What problem does RiseUp solve?

People dismiss alarms without getting up, ignore check-ins, and miss task reminders. One-size-fits-all alarms and static reminders do not adapt to what actually works for each person.

Who are the target users?

- People who struggle to wake (ADHD, depression, sleep issues)
- Students with irregular schedules
- Professionals with critical morning routines
- Anyone who needs accountable task reminders

Why an AI agent is appropriate

Waking and reminding require **multi-step decisions under uncertainty**: choose a channel, try it, observe the outcome, try another if it failed, and learn over time. That is agent behavior (observe → decide → tool use → re-observe), not a single chat completion.

2. Solution: RiseUp Agent

RiseUp is an intelligent alarm and task-reminder system with:

- **GUI:** React web app for alarms, tasks, insights, history, preferences
- **CLI:** `riseup` commands over the same FastAPI backend (major functionality)
- **Agent:** `AgentController` loop with LLM decisions and tool execution
- **Memory:** `UserProfile` + `EscalationLog` (and reminder logs)

- **Tools:** Twilio (call/SMS), Alexa, Firebase push, camera/object-detection for verification

What the agent can do

- Interpret natural-language reminder requests
- Decide escalation method when an alarm/check-in is ignored
- Rank simultaneous reminders
- Propose personalized setting changes from learned stats
- Recover when a tool fails by choosing a fallback (or the deterministic ladder)

AI / LLM models

Provider	Model	Default use
Anthropic	claude-sonnet-4-5	Escalation, recommendations, prioritization
OpenAI	gpt-4o	Natural-language reminder parsing

Both implement `LLMProvider` (Strategy). `LLMClient` selects the configured provider.

How the AI interacts with the rest of the system

```

GUI / CLI → Controllers → AgentController / domain services
                                     |
                                     ├── PromptBuilder → LLMClient → LLMProvider
                                     ├── ResponseParser (validate JSON)
                                     ├── ToolManager → Tool adapters (Twilio, Alexa, Firebase)
                                     └── UserProfile / EscalationLog (memory)

```

Deterministic parts (scheduling, math/trivia validation, DB, tool HTTP calls) never depend on the LLM succeeding; AI failure triggers documented fallbacks.

3. Assumptions & Constraints

Assumptions

- Users have internet connectivity
- Users want to wake up (not coercive)
- At least one escalation channel is enabled
- Platform: web + CLI (Windows, macOS, Linux)

Constraints

- Minimal personal data; explainable AI decisions (F14)
- Escalation SMS/calls must respect consent / quiet hours
- Emergency contact only after explicit consent (F06)

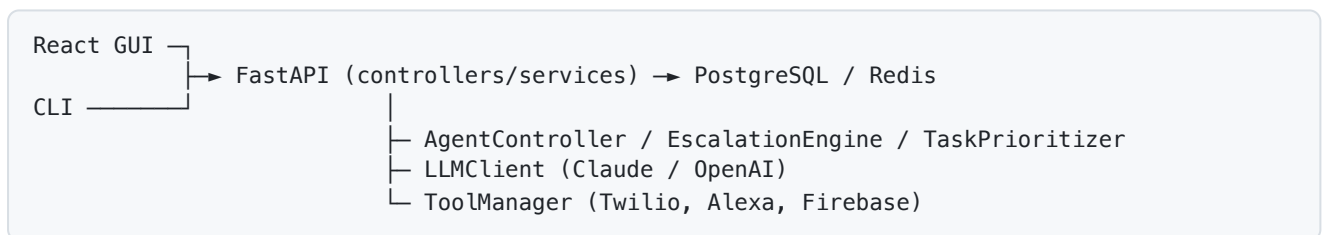
4. Main Features (F01–F14)

Detail (inputs, outputs, workflows, errors) lives in `feature-specification.md`. Summary:

ID	Feature	Type
F01	Alarm Management	Deterministic
F02	Verification Challenge	Deterministic
F03	Post-Wake Check-in	Deterministic
F04	AI Escalation Decision	AI / Hybrid
F05	Multi-Channel Escalation Execution	Deterministic
F06	Emergency Contact & Escalation Preferences	Deterministic
F07	Natural-Language Reminder Creation	Hybrid
F08	Reminder Management	Deterministic
F09	AI Reminder Prioritization	AI / Hybrid
F10	Reminder Escalation Levels	Deterministic
F11	Behavioral Learning & Profile Update	Deterministic
F12	Sleep & Completion Analytics Dashboard	Deterministic
F13	AI Personalized Recommendations	AI
F14	History & Decision Explanation	Hybrid

Not counted as features: login/logout, generic settings shell, “having a web app,” bare scheduler infra.

5. Overall Architecture



UML: [uml-diagrams/](#) — class, use-case, sequence (SD-01...SD-07). Patterns: [design-patterns.md](#).

6. Out of Scope (Stage 1–2)

- Wearables / biometrics
- Google/Outlook calendar sync
- Multi-user family accounts
- Training a custom ML model (use hosted LLMs)

7. Success Criteria

- Alarm cannot be dismissed without verification; check-in verifies wakefulness
- Escalation is agent-driven with deterministic fallback
- CLI and GUI both reach major features
- Learning updates profiles used by later decisions

- Decisions are explainable in history (F14)
- Deterministic components unit-testable; agent behavior testable later with KUMA

RiseUp - Feature Specification & CLI Design

Chapter 2 — source: docs/stage-1-design/feature-specification.md

Chapter overview. Chapter 2 is the **canonical definition of the 14 features**. Each feature lists the same eight fields: description, GUI interaction, CLI command, input, output, AI involvement, expected workflow (class/method chain) and error/alternative cases, followed by its main classes. Section 3 specifies the **CLI**: the command table, global options and exit codes, an example session, the CLI classes, and the Command and Strategy patterns it adds, with a CLI class diagram and the `riseup remind` sequence diagram in Mermaid.

How to read it: use the summary table in Section 1 to find a feature, then go to its block in Section 2. The *Expected Workflow* line of each feature matches the sequence diagrams in [Chapter 7](#).

This document replaces the 5 feature groups in `project-scope.md` with **14 distinct features**, each specified with the 8 fields required by the Stage 1 instructions. It also adds the **CLI** (required by the instructions) as a second interface to the same system.

Conventions:

- **Type:** Deterministic / AI / Hybrid (AI decides, deterministic code validates and executes, per AD-02).
- **GUI:** React web app. **CLI:** `riseup` command (see Section 3).
- Class names follow `feature-to-design-mapping.md`. **New classes** introduced here are marked **(new)**.

1. Feature Summary

ID	Feature	Type	Use Case
F01	Alarm Management	Deterministic	UC-01
F02	Verification Challenge	Deterministic	UC-02
F03	Post-Wake Check-in	Deterministic	UC-02
F04	AI Escalation Decision	AI (Hybrid with fallback)	UC-03
F05	Multi-Channel Escalation Execution	Deterministic	UC-03
F06	Emergency Contact & Escalation Preferences	Deterministic	UC-08 (new)
F07	Natural-Language Reminder Creation	Hybrid	UC-04
F08	Reminder Management	Deterministic	UC-04, UC-06 (new)
F09	AI Reminder Prioritization	AI (Hybrid with fallback)	UC-05
F10	Reminder Escalation Levels	Deterministic	UC-05
F11	Behavioral Learning & Profile Update	Deterministic	UC-03, UC-05
F12	Sleep & Completion Analytics Dashboard	Deterministic	UC-07 (new)
F13	AI Personalized Recommendations	AI	UC-07 (new)
F14	History & Decision Explanation	Hybrid	UC-09 (new)

Not counted as features (per the instructions): login/logout, settings screens, "web app" itself, scheduler infrastructure.

2. Detailed Feature Specifications

F01 - Alarm Management

- **Description:** Create, edit, enable/disable, and delete alarms with time, label, sound, vibration intensity, repeat pattern, snooze setting, and verification task type. Limit of 20 alarms per user.
- **User Interaction (GUI):** Dashboard → "New Alarm" → alarm form → "Save Alarm". Existing alarms show a toggle, Edit, and Delete button.
- **CLI:** `riseup alarm add|list|edit|delete|toggle`
- **Input:** time (HH:MM), label, sound, vibration level, repeat pattern, snooze on/off, verification task type.
- **Output:** persisted `Alarm`, scheduled trigger job, confirmation "Alarm set for 07:00".
- **AI Involvement:** None (deterministic).
- **Expected Workflow:** UI/CLI → `AlarmController.create_alarm()` → `Alarm.validate()` → `AlarmRepository.save()` → `AlarmScheduler.schedule_alarm()` → confirmation returned.
- **Error/Alternative Cases:** invalid time format (inline error); 20-alarm limit reached (prompt to delete one); DB failure (retry message, nothing scheduled); user cancels (no data saved); edit of a running alarm (rejected until it finishes).
- **Main classes:** `Alarm`, `AlarmScheduler`, `AlarmRepository` (**new**), `UserPreferences`.

F02 - Verification Challenge

- **Description:** When an alarm fires, it cannot be dismissed until the user completes a challenge: photo object detection, math puzzle, trivia, or custom task. Wrong attempts reset the challenge; repeated camera failure falls back to a math puzzle.
- **User Interaction (GUI):** Full-screen alarm page with disabled Dismiss button and a task panel (camera, answer field, or "Mark completed").
- **CLI:** `riseup alarm verify <alarm-id> --answer 42` (math/trivia/custom). Photo verification is GUI-only (needs camera); the CLI reports this clearly.
- **Input:** task answer, photo, or completion confirmation.
- **Output:** pass/fail result; on pass, alarm stops and completion time is logged.
- **AI Involvement:** None. Photo check uses a vision library and answers are validated deterministically (AD-02).
- **Expected Workflow:** `AlarmScheduler.trigger_alarm()` → `Alarm.trigger()` → `VerificationTaskFactory.create()` (**new**) → task shown → `VerificationTask.validate(response)` → on success `Alarm.stop()` and `CheckIn` scheduled.
- **Error/Alternative Cases:** wrong answer (retry, alarm keeps ringing); blurry photo (retry); camera unavailable (fallback to `MathTask`); user tries to close page (blocked); snooze pressed (allowed up to 3 times if enabled); no response for 2 min (hand over to F04).
- **Main classes:** `VerificationTask` (abstract), `ObjectDetectionTask`, `MathTask`, `TriviaTask`, `CustomTask`, `VerificationTaskFactory` (**new**).

F03 - Post-Wake Check-in

- **Description:** 5-10 minutes after a successful verification, the system asks the user to confirm they are still awake, preventing "verify then go back to sleep".
- **User Interaction (GUI):** Push notification/banner "Are you still awake?" with a Confirm button.
- **CLI:** `riseup checkin confirm`
- **Input:** user confirmation within 5 minutes.
- **Output:** check-in recorded as confirmed or missed; missed triggers F04.
- **AI Involvement:** None.
- **Expected Workflow:** `CheckIn.schedule()` → `ToolManager.send_push_notification()` → wait → `CheckIn.record_response()` or timeout → `EscalationEngine.decide_escalation()`.
- **Error/Alternative Cases:** push delivery fails (fall back to SMS); user confirms late (logged as late, no escalation); user has disabled check-ins (skipped, logged).
- **Main classes:** `CheckIn`, `ToolManager`, `FirebaseClient`.

F04 - AI Escalation Decision

- **Description:** When the user ignores the alarm or misses the check-in, the agent reads the user's history and decides which escalation method to use next (call, SMS, Alexa, emergency contact), when, and with what message. The decision includes reasoning and confidence.
- **User Interaction (GUI):** Mostly automatic. The user sees "Escalation: SMS sent" on the dashboard and can open "Why?" (see F14).
- **CLI:** `riseup escalation explain <id>`
- **Input:** `UserProfile` stats, recent `EscalationLog` entries, time of day, available methods.
- **Output:** validated decision JSON: `{method, reasoning, confidence, fallback}`.
- **AI Involvement: AI.** `LLMClient` sends a structured prompt and parses JSON. Output is validated (method must be in the enabled list).
- **Expected Workflow:** `AgentController.run("escalate")` → `EscalationEngine.decide_escalation()` → `PromptBuilder.build_escalation_prompt()` → `LLMClient.send_request()` → `ResponseParser.parse_decision()` → validate → return decision to F05.
- **Error/Alternative Cases:** API timeout/error (rule-based fallback ladder UX-01); malformed JSON (one retry, then fallback); LLM picks a disabled method (rejected, fallback used); all methods disabled (log "missed alarm", no escalation); multiple alarms at once (highest priority first).
- **Main classes:** `AgentController`, `EscalationEngine`, `LLMClient`, `LLMProvider`, `PromptBuilder`, `ResponseParser`, `UserProfile`, `EscalationLog`.

F05 - Multi-Channel Escalation Execution

- **Description:** Executes the chosen escalation through external services and records the outcome. If a channel fails, the next fallback channel is tried.
- **User Interaction (GUI):** Status line on the alarm card: "Calling... no answer → SMS sent → confirmed".
- **CLI:** `riseup escalation status <alarm-id>`
- **Input:** decision from F04, user phone number, contact details.
- **Output:** call/SMS/Alexa/contact message sent; `EscalationLog` entry (method, time, response, outcome).
- **AI Involvement:** None (AI decides in F04, code executes).

- **Expected Workflow:** `ToolManager.execute_escalation(method)` → `TwilioClient.make_call()` / `send_sms()` / `AlexaClient.send_alert()` → wait for response → `EscalationLog.record_escalation()` → `UserProfile.update_learning_data()`.
- **Error/Alternative Cases:** call unreachable (wait 30 s, fallback SMS); no Alexa device (skip); emergency contact unreachable (3 attempts, then log failure and notify user); user replies "do not call" (preference updated); whole chain fails (dashboard shows "Alarm unanswered").
- **Main classes:** `ToolManager`, `Tool` (interface) **(new)**, `TwilioClient`, `AlexaClient`, `EscalationLog`.

F06 - Emergency Contact & Escalation Preferences

- **Description:** User manages which escalation channels are enabled, phone numbers, quiet hours, and an emergency contact who is only used as a last resort. Enforces privacy/consent: the contact must be explicitly confirmed before being used.
- **User Interaction (GUI):** Settings → Escalation: toggles per channel, contact form, "Send test message".
- **CLI:** `riseup prefs show|set`, `riseup contact add|remove|test`
- **Input:** enabled channels, phone number, contact name/phone, quiet hours.
- **Output:** saved `UserPreferences`; test message result.
- **AI Involvement:** None.
- **Expected Workflow:** `PreferencesController.update()` → `UserPreferences.validate()` → `EmergencyContact.request_consent()` **(new)** → save → optional test via `ToolManager`.
- **Error/Alternative Cases:** invalid phone format (error); contact has not consented (contact excluded from escalation); all channels disabled (warning that alarms will not escalate).
- **Main classes:** `UserPreferences`, `EmergencyContact` **(new)**, `ToolManager`.

F07 - Natural-Language Reminder Creation

- **Description:** User types (or speaks via Alexa) "Remind me to call mom tomorrow at 9am". The agent extracts task, date/time, priority, and category, then asks for confirmation before saving.
- **User Interaction (GUI):** Quick-add text box on the task dashboard; parsed result shown as an editable confirmation card.
- **CLI:** `riseup remind "call mom tomorrow at 9am"`
- **Input:** free-text sentence plus user timezone and current time.
- **Output:** proposed `Reminder` (title, due time, priority, category); after confirmation, a saved reminder.
- **AI Involvement: Hybrid.** `LLMClient` extracts fields as JSON. Deterministic code validates date/time and creates the reminder.
- **Expected Workflow:** `NLCommandParser.parse(text)` **(new)** → `PromptBuilder.build_parse_prompt()` → `LLMClient.send_request()` → `ResponseParser.parse_reminder()` → `Reminder` draft → user confirms → `ReminderFactory.create()` **(new)** → `ReminderScheduler.schedule_reminder()`.
- **Error/Alternative Cases:** ambiguous input (agent asks a clarifying question, e.g. "At what time?"); time in the past (offer "tomorrow instead?"); LLM unavailable (open the manual form pre-filled with the text); malformed JSON (retry once, then manual form); user rejects parse (edit fields).
- **Main classes:** `NLCommandParser`, `LLMClient`, `PromptBuilder`, `ResponseParser`, `ReminderFactory`, `Reminder`.

F08 - Reminder Management

- **Description:** Create, edit, complete, snooze, and delete reminders by form; supports recurring reminders and multiple reminder times per task.
- **User Interaction (GUI):** Task dashboard list with priority/category filters; Complete, Snooze, Edit, Delete buttons; "New Reminder" form.
- **CLI:** `riseup task add|list|edit|done|snooze|delete`
- **Input:** title, due date/time, priority, category, repeat pattern, notification type.
- **Output:** persisted `Reminder` ; updated schedule; completion timestamp on completion.
- **AI Involvement:** None.
- **Expected Workflow:** `ReminderController` → `Reminder.validate()` → `ReminderRepository.save()` (**new**) → `ReminderScheduler.schedule_reminder()` .
- **Error/Alternative Cases:** blank title (error); invalid time; past time (offer tomorrow); edit of recurring series (ask: this occurrence or all); DB failure (retry).
- **Main classes:** `Reminder` , `ReminderScheduler` , `ReminderRepository` (**new**), `ReminderFactory` .

F09 - AI Reminder Prioritization

- **Description:** When several reminders are due at the same time, the agent ranks them using priority, time of day, and the user's completion rates by category, so the most useful one is sent first.
- **User Interaction (GUI):** Automatic. The notification order is visible, and "Why this order?" is available in History (F14).
- **CLI:** `riseup task priority-preview` (shows the ranking for currently due tasks)
- **Input:** list of due reminders, `UserProfile.task_completion_rates` , time of day.
- **Output:** ordered list of reminders with short reasoning.
- **AI Involvement:** AI with deterministic fallback.
- **Expected Workflow:** `ReminderScheduler.on_due()` → `TaskPrioritizer.prioritize_reminders()` → `LLMClient.send_request()` → `ResponseParser.parse_ranking()` → validate that every reminder appears exactly once → send in order.
- **Error/Alternative Cases:** only one reminder due (skip AI); LLM failure or invalid ranking (fallback sort: priority → category → due time); LLM drops or invents a reminder (ranking rejected, fallback used).
- **Main classes:** `TaskPrioritizer` , `LLMClient` , `ResponseParser` , `UserProfile` .

F10 - Reminder Escalation Levels

- **Description:** An unanswered reminder is re-sent at +5 min (escalation tone) and +15 min (final, urgent tone), then marked missed. Stops as soon as the task is completed.
- **User Interaction (GUI):** Notification sequence with Complete/Snooze/"Already done" actions; missed tasks get a red flag on the dashboard.
- **CLI:** `riseup task done <id>` stops the sequence.
- **Input:** reminder, user responses.
- **Output:** up to 3 notifications; completion or "missed" status logged.
- **AI Involvement:** None.
- **Expected Workflow:** `ReminderScheduler.send_primary_reminder()` → wait → `send_escalation_reminder()` → wait → `send_final_reminder()` → `Reminder.mark_missed()` . Observers (`DashboardNotifier` , `ReminderLogger`) react to status changes.
- **Error/Alternative Cases:** user completes early (sequence cancelled); device offline (notification queued); notifications disabled (scheduled for next login); user snoozes (sequence restarts at new time).

- **Main classes:** `ReminderScheduler`, `Reminder`, `ToolManager`, `ReminderObserver` **(new)**.

F11 - Behavioral Learning & Profile Update

- **Description:** Nightly job aggregates logs into `UserProfile`: escalation success rate per method, average wake time, completion rate by category/hour, fastest verification task type.
- **User Interaction (GUI):** No direct action. Results appear in F12/F13.
- **CLI:** `riseup profile show`, `riseup profile recompute`
- **Input:** `EscalationLog`, alarm logs, reminder logs.
- **Output:** updated `UserProfile` statistics.
- **AI Involvement:** None (pure statistics, which makes it fully unit-testable).
- **Expected Workflow:** `LearningEngine.analyze_behavior()` → `AnalyticsEngine.compute_metrics()` → `UserProfile.update_learning_data()` → cache refresh.
- **Error/Alternative Cases:** too little data (keep defaults, mark "insufficient data"); job fails (previous profile kept, error logged); corrupted log rows (skipped and counted).
- **Main classes:** `LearningEngine`, `AnalyticsEngine`, `UserProfile`, `EscalationLog`.

F12 - Sleep & Completion Analytics Dashboard

- **Description:** Charts and numbers showing wake times, verification success, escalation effectiveness by method, and task completion by category over 7/30/90 days.
- **User Interaction (GUI):** "Insights" page with date-range selector and charts.
- **CLI:** `riseup stats --range 30d [--json]`
- **Input:** date range.
- **Output:** charts (GUI) or table/JSON (CLI).
- **AI Involvement:** None.
- **Expected Workflow:** `AnalyticsController.get_stats(range)` → `AnalyticsEngine.query()` → `OutputFormatter` (CLI) or chart components (GUI).
- **Error/Alternative Cases:** no data in range (empty-state message); invalid range (error); slow query (cached result served).
- **Main classes:** `AnalyticsEngine`, `UserProfile`, `AnalyticsController` **(new)**.

F13 - AI Personalized Recommendations

- **Description:** The agent turns the user's statistics into concrete suggestions, e.g. "SMS works 40% for you, calls 95%: make calls your first escalation" or "You miss health tasks at 6 PM: move them to 8 AM". The user accepts or dismisses each suggestion.
- **User Interaction (GUI):** "Recommendations" panel with Accept/Dismiss buttons. Accept applies the setting change.
- **CLI:** `riseup recommend [--apply <n>]`
- **Input:** `UserProfile` metrics and current settings.
- **Output:** up to 5 suggestions, each with a proposed setting change and rationale.
- **AI Involvement:** AI. Suggestions are validated against allowed settings before being shown; applying a change is deterministic.
- **Expected Workflow:** `RecommendationService.generate()` **(new)** → `PromptBuilder.build_insight_prompt()` → `LLMClient.send_request()` → `ResponseParser.parse_recommendations()` → validate each change against the settings schema → return; on accept `UserPreferences.update()`.

- **Error/Alternative Cases:** insufficient data (no suggestions, explains why); LLM failure (show rule-based tips only); suggestion invalid or unsafe (e.g. disables all escalation, so it is dropped); repeated dismissals (suggestion type suppressed).
- **Main classes:** `RecommendationService`, `LLMClient`, `UserProfile`, `UserPreferences`.

F14 - History & Decision Explanation

- **Description:** Searchable history of alarms, verifications, escalations, and reminders. Each AI decision stores its reasoning and confidence, so the user can see why an escalation or ordering happened (satisfies the "AI decisions must be explainable" constraint).
- **User Interaction (GUI):** "History" page with filters (type, date); click an entry to see "Why?" details.
- **CLI:** `riseup history [--type escalation] [--since 7d]`, `riseup escalation explain <id>`
- **Input:** filters; entry ID.
- **Output:** list of events; for AI decisions, the reasoning, confidence, inputs used, and whether a fallback was used.
- **AI Involvement: Hybrid.** Display is deterministic; the reasoning text was stored from F04/F09.
- **Expected Workflow:** `HistoryController.list(filter)` → `EscalationLog` / `ReminderLog` queries → formatted; `explain(id)` → load stored decision record.
- **Error/Alternative Cases:** entry not found (404 message); decision made by fallback (clearly labelled "rule-based fallback, LLM unavailable"); large history (paginated).
- **Main classes:** `HistoryController` (new), `EscalationLog`, `ReminderLog` (new).

3. CLI Design

3.1 Requirement and approach

The instructions require a CLI through which users can reach the **major functionality**. RiseUp's CLI is a thin client over the **same FastAPI backend** the GUI uses. This keeps one source of truth (the scheduler and database live in the server) and avoids duplicating business logic.



Suggested implementation: Python with **Typser** (or `argparse`), HTTP via `httpx`.

3.2 Command overview

Command	Feature	Example
<code>riseup alarm</code> <code>add/list/edit/delete/toggle</code>	F01	<code>riseup alarm add 07:00 --task math --repeat weekdays</code>
<code>riseup alarm verify <id> --answer N</code>	F02	<code>riseup alarm verify 12 --answer 42</code>
<code>riseup checkin confirm</code>	F03	<code>riseup checkin confirm</code>
<code>riseup escalation explain/status <id></code>	F04, F05, F14	<code>riseup escalation explain 88</code>
<code>riseup prefs show/set</code> , <code>riseup contact</code> <code>add/remove/test</code>	F06	<code>riseup contact add "Sam" +14165550123</code>

Command	Feature	Example
<code>riseup remind "<sentence>"</code>	F07	<code>riseup remind "call mom tomorrow at 9am"</code>
<code>riseup task</code> <code>add/list/edit/done/snooze/delete</code>	F08, F10	<code>riseup task done 5</code>
<code>riseup task priority-preview</code>	F09	<code>riseup task priority-preview</code>
<code>riseup profile show/recompute</code>	F11	<code>riseup profile show --json</code>
<code>riseup stats --range 30d</code>	F12	<code>riseup stats --range 30d</code>
<code>riseup recommend [--apply n]</code>	F13	<code>riseup recommend --apply 2</code>
<code>riseup history --type ... --since ...</code>	F14	<code>riseup history --type escalation --since 7d</code>

Global options: `--json` (machine-readable output), `--profile <name>`, `--server <url>`, `--token` (API auth). Exit codes: `0` success, `1` user/input error, `2` server or service error.

Photo verification and live alarm ringing remain GUI-only (they need a camera/audio). The CLI documents this and never silently skips it.

3.3 Example session

```
$ riseup remind "call mom tomorrow at 9am"
Parsed: "Call mom" | 2026-10-01 09:00 | priority: medium | category: personal
Save this reminder? [Y/n] y
Reminder #14 scheduled for 2026-10-01 09:00

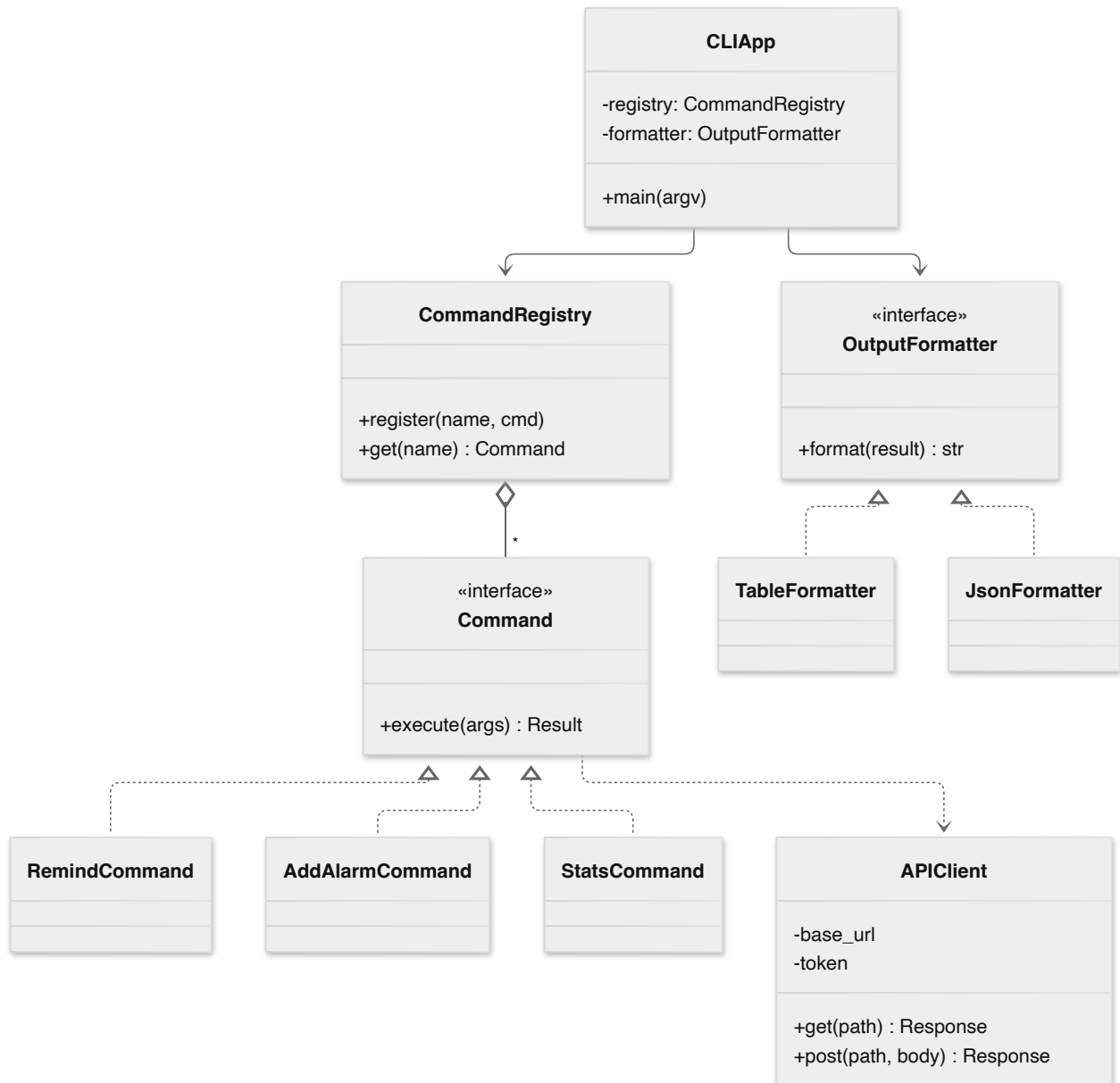
$ riseup escalation explain 88
Escalation #88 (2026-09-29 07:04)
Method chosen : call          Confidence: 0.91
Reasoning      : Calls woke you in 95% of past escalations; SMS 40%.
Decision by    : LLM (Claude)  Fallback used: no
Outcome        : confirmed after 45 s
```

3.4 CLI classes (new) and patterns

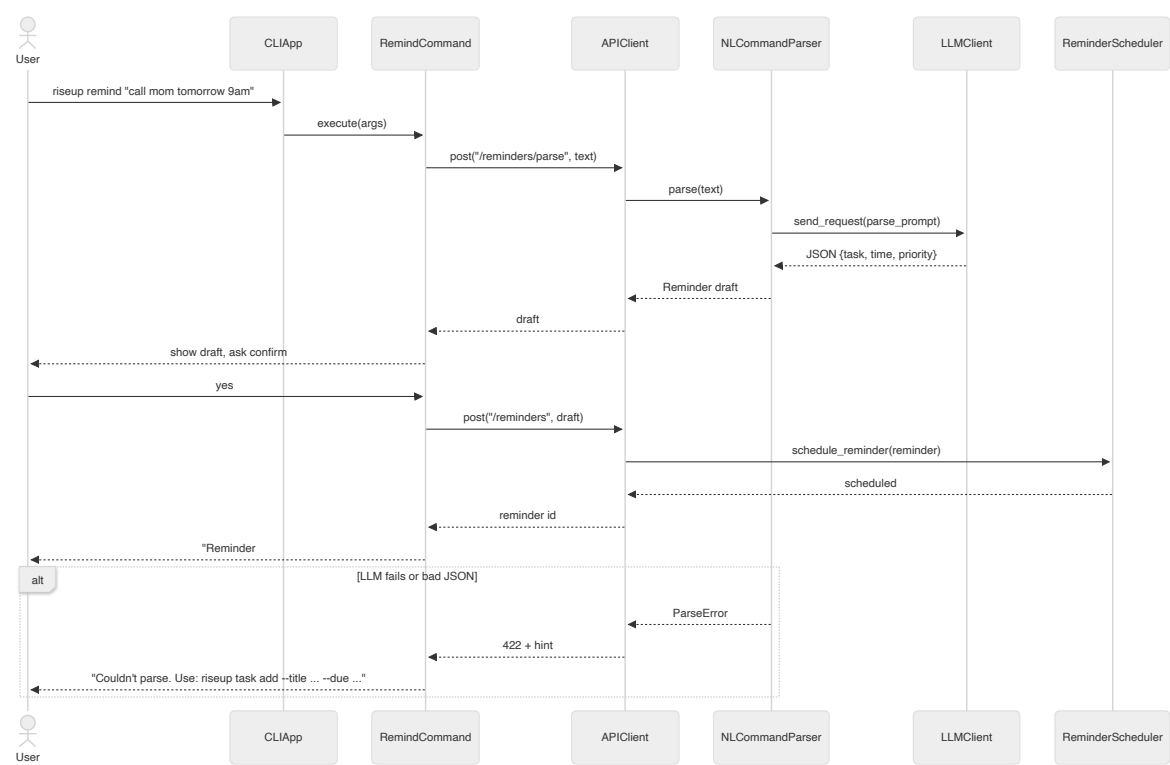
Class	Responsibility
<code>CLIApp</code>	Entry point; parses arguments, selects a command, sets exit code
<code>Command</code> (interface)	<code>execute(args) -> Result</code>
<code>AddAlarmCommand</code> , <code>ListTasksCommand</code> , <code>RemindCommand</code> , <code>StatsCommand</code> , ...	Concrete commands, one per CLI action
<code>CommandRegistry</code>	Maps command names to <code>Command</code> objects
<code>APIClient</code>	Sends HTTP requests to the backend; handles auth and errors
<code>OutputFormatter</code> (interface)	<code>format(result)</code> ; implemented by <code>TableFormatter</code> and <code>JsonFormatter</code>

Patterns this adds:

- **Command:** each CLI action is an object with `execute()` . New commands are added without changing `CLIApp` , and commands can be tested in isolation.
- **Strategy:** `OutputFormatter` implementations (`TableFormatter` / `JsonFormatter`) selected by `--json` .
- **Note:** `APIClient` hides HTTP/auth details (optional small Facade; not counted toward the five-pattern minimum).



3.5 CLI sequence (riseup remind "...", F07)



4. Alignment with other Stage 1 docs

This file is the **canonical F01–F14** definition. Other docs must match it:

Doc	Expected content
project-scope.md	F01–F14 summary, CLI + GUI, models, agent loop
use-cases.md	UC-01–UC-09 with Related Feature(s); no UC-03→UC-05 link
feature-to-design-mapping.md	Traceability + Task 4 for all 14 features
design-patterns.md	Strategy, Adapter, Command, Observer, Factory Method
design-decisions.md	AD-05 AgentController; UX-01 = fallback ladder; decide_escalation on EscalationEngine
uml-diagrams/	Class diagram includes new classes; SD-01...SD-07 per mapping ID table

Agent involvement: F04/F05/F07/F09/F13 go through AgentController observe→decide→act→re-observe when tools are used. Memory: UserProfile + EscalationLog .

Models: claude-sonnet-4-5 (default escalation/recommendations), gpt-4o (default NL parse), both via LLMPProvider .

Patterns in CLI: Command + Strategy (OutputFormatter). APIClient may act as a small HTTP Facade but is not required for the five-pattern minimum.

RiseUp - Use Cases

Chapter 3 — source: docs/stage-1-design/use-cases.md

Chapter overview. Chapter 3 gives the nine use cases in a standard template: ID, name, related features, primary and secondary actors, goal, preconditions, trigger, main success scenario, alternative flows, exception flows, postconditions and notes. UC-01 – UC-05 cover the core alarm, escalation and reminder flows in full detail. UC-06 – UC-09 cover reminder management, analytics and recommendations, preferences and emergency contact, and history and explanations. The chapter ends with the use-case diagram (Mermaid), the dependencies between use cases and a summary table.

Note: UC-03 (escalation) and UC-05 (reminder prioritisation) both use the LLM but are independent flows.

Use Case Overview

- 1. **UC-01:** Configure and Set an Alarm — F01
- 2. **UC-02:** Alarm Verification & Check-in — F02, F03
- 3. **UC-03:** AI Escalation — F04, F05
- 4. **UC-04:** Create a Task Reminder (form or NL) — F07, F08
- 5. **UC-05:** Send Prioritized Task Reminders — F09, F10
- 6. **UC-06:** Manage Existing Reminders — F08
- 7. **UC-07:** View Analytics & Recommendations — F11, F12, F13
- 8. **UC-08:** Manage Escalation Preferences & Emergency Contact — F06
- 9. **UC-09:** View History & Decision Explanation — F14

External actors (shown on the use-case diagram): **User**, **System/Scheduler**, **LLM Service** (Claude / OpenAI), **Twilio**, **Alexa**, **Camera / Vision**, **Firebase** (push).

UC-01: User Configures and Sets an Alarm

Use Case ID

UC-01

Use Case Name

Configure and Set Alarm

Related Feature(s)

F01 — Alarm Management

Primary Actor

User

Secondary Actors

- System (Alarm Scheduler)

Goal

User creates a new alarm with customized settings and activates it for a specific time.

Preconditions

- User is logged into RiseUp
- User has access to the web application
- System has current time and timezone information

Trigger

User clicks "New Alarm" or "Add Alarm" button on the dashboard.

Main Success Scenario

1. System displays alarm creation form
2. User enters alarm time (HH:MM)
3. User selects alarm sound/tone from library
4. User sets vibration intensity (low/medium/high)
5. User selects verification task type:
 - Object Detection (take photo)
 - Math Puzzle (solve equation)
 - Trivia Question (answer question)
 - Custom Task (user-defined)
6. User optionally sets label/name (e.g., "Work Alarm", "Gym Alarm")
7. User configures repeat pattern (once, daily, weekdays, custom)
8. User optionally sets snooze availability (enable/disable)
9. User reviews settings
10. User clicks "Save Alarm"
11. System validates alarm configuration
12. System schedules alarm in the database
13. System confirms "Alarm set for [time]"
14. Alarm appears on dashboard as active

Alternative Flows

A1: User Cancels Alarm Creation

- At any step, user clicks "Cancel"
- System discards all entries
- User returns to dashboard
- No alarm is created

A2: Invalid Time Entry

- User enters invalid time format
- System displays error message: "Please enter time in HH:MM format"
- User re-enters time
- Continue from step 2 in main flow

A3: User Selects Custom Verification Task

- At step 5, user selects "Custom Task"
- System shows text input for custom task description
- User enters task (e.g., "Do 10 pushups")
- User confirms custom task
- Continue with step 6 in main flow

A4: User Edits Existing Alarm

- User clicks "Edit" on an existing alarm
- System loads alarm settings into form
- User modifies settings
- User clicks "Save"
- System updates alarm in database
- Updated alarm replaces old one

Exception Flows

E1: System Fails to Save Alarm

- User clicks "Save Alarm"
- System attempts to save but database connection fails
- System displays error: "Failed to save alarm. Please try again."
- User clicks "Retry"
- System attempts save again
- If successful, continue with step 13 in main flow
- If failed again, offer to save offline or contact support

E2: User Exceeds Maximum Alarms

- User has 20 alarms already set
- System displays: "Maximum alarms reached. Delete an alarm to add another."
- User is directed to alarm management screen
- User must delete an alarm first before creating new one

Postconditions

- Alarm is stored in the database
- Alarm is scheduled to trigger at specified time
- Alarm appears on user's dashboard as active
- User receives confirmation notification

Notes

- Alarm persists across sessions (if user logs out/closes app)
 - User can edit or delete alarm at any time before it triggers
 - Multiple alarms can be set for the same time
-

UC-02: Alarm Triggers and User Completes Verification Task

Use Case ID

UC-02

Use Case Name

Alarm Triggers and Complete Verification to Dismiss

Related Feature(s)

F02 — Verification Challenge; F03 — Post-Wake Check-in

Primary Actor

User

Secondary Actors

- System (Alarm Engine, Scheduler)
- Camera / Vision (for object-detection tasks)
- Firebase (check-in push)

Goal

When alarm triggers, user must complete a verification task to prove they are awake before alarm can be dismissed.

Preconditions

- Alarm has been set and is enabled (from UC-01)
- Current time equals alarm time
- User's device is connected to internet
- User is nearby to receive alarm

Trigger

System's clock reaches the scheduled alarm time.

Main Success Scenario

1. System plays alarm sound at maximum volume
2. System activates vibration at configured intensity

3. System displays alarm screen with:
 - Alarm name/label
 - Current time
 - Assigned verification task
 - "Cannot dismiss - Complete task" message
 - Dismiss button is disabled/grayed out
4. User wakes up and responds to alarm
5. User reads the verification task
6. Based on task type:
 - **Object Detection:** User taps "Take Photo" → Camera opens → User takes photo of self/room → System analyzes photo → Verifies object present
 - **Math Puzzle:** System displays equation → User enters answer → System validates answer
 - **Trivia:** System displays question → User selects answer → System validates answer
 - **Custom Task:** System displays task description → User marks as "Completed" when done
7. System validates task completion
8. If valid: Task completion succeeds
9. System stops alarm sound/vibration
10. System displays "Task completed! Rest well." message
11. System records completion time
12. System schedules check-in reminder for 5-10 minutes later
13. Alarm screen closes, user returns to dashboard
14. System stores alarm completion log

Alternative Flows

A1: User Completes Verification Task Incorrectly

- User attempts verification task (e.g., takes blurry photo, answers math wrong)
- System validation fails
- System displays: "Task not valid. Please try again."
- Task resets for another attempt
- User must try again
- Alarm sound/vibration continues
- Continue from step 5 in main flow (user retries task)

A2: User Snooze Is Available

- User has enabled snooze in alarm settings (UC-01)
- A "Snooze 10 min" button appears on alarm screen
- If user clicks snooze:
 - Alarm pauses for 10 minutes
 - Alarm sound/vibration stops temporarily
 - At end of 10 min, alarm triggers again
 - Return to step 1 in main flow
 - Note: Limited snooze attempts (e.g., max 3 snoozes)

A3: User Disables Alarm (Invalid Attempt)

- User tries to close/swipe away alarm screen
- System prevents dismissal: "Complete task to dismiss alarm"

- Alarm sound/vibration continues
- Alarm remains on screen
- Return to step 5 in main flow

Exception Flows

E1: Verification Task Fails Unexpectedly

- User attempts photo object detection
- System camera fails or photo upload fails
- System displays: "Camera error. Please try again or contact support."
- User can retry with same task
- If repeated failures, offer fallback task (math puzzle instead)

E2: User Doesn't Respond to Alarm

- Alarm has been triggering for 2+ minutes with no user action
- System detects inactivity
- System transitions to UC-03 (Escalation)
- Check-in is skipped
- Escalation begins

E3: Check-in Reminder After Task Completion

- User completes verification task at 7:05 AM
- System schedules check-in for 7:10-7:15 AM
- Check-in is sent as:
 - Notification push: "Are you still awake? Tap to confirm"
 - Or SMS: "RiseUp check-in: Reply YES to confirm you're awake"
- If user responds within 5 minutes → Success (log recorded)
- If user doesn't respond → Transition to UC-03 (Escalation)

Postconditions

- Alarm sound/vibration is stopped
- Verification task is marked as completed
- Completion time is recorded in database
- Check-in reminder is scheduled
- Alarm log entry is created for analytics
- User dashboard shows alarm as "completed"

Notes

- Verification task prevents casual dismissal
- Check-in 5-10 min later verifies actual wakefulness
- Failed check-in triggers escalation (UC-03)
- System learns which task types user completes fastest
- Alarm logs help identify patterns (sleep quality, task effectiveness)

UC-03: User Ignores Alarm or Verification → AI Escalates

Use Case ID

UC-03

Use Case Name

AI Escalation When User Doesn't Respond

Related Feature(s)

F04 — AI Escalation Decision; F05 — Multi-Channel Escalation Execution

Primary Actor

System (AgentController / EscalationEngine)

Secondary Actors

- User (recipient of escalation)
- LLM Service (Claude `claude-sonnet-4-5` / OpenAI `gpt-4o`)
- Twilio (calls / SMS)
- Alexa (alerts)
- Firebase (optional push)

Goal

When user doesn't respond to alarm or verification task, system intelligently escalates using AI to determine the best next action.

Preconditions

- Alarm has triggered (from UC-02)
- User has not completed verification task within timeout (e.g., 2 minutes)
- OR user completed task but didn't respond to check-in (5-10 min check-in)
- User's escalation preferences are configured
- System has access to at least one escalation tool (call, SMS, Alexa, etc.)
- User profile and past escalation history exist in database

Trigger

Either:

- Alarm triggered 2+ minutes ago with no verification task completion
- OR Check-in reminder sent but no response within 5 minutes

Main Success Scenario

Phase 1: Gather User Context

1. System retrieves user profile:

- Past alarm behaviors (response times)
 - Which escalation methods worked previously
 - Current time and day of week
 - User's typical wake-up patterns
 - Sleep quality history
2. System accesses escalation history log:
- Previous escalations and outcomes
 - Which method successfully woke user
 - Time of day patterns

Phase 2: AI Decision Making

3. System sends request to Claude LLM with context:
- "User [name] did not respond to alarm at [time]"
 - "Previous escalations that worked: [list]"
 - "Failed escalations: [list]"
 - "User's availability: [phone/SMS/Alexa enabled]"
 - "Available escalation methods: [call/SMS/Alexa/emergency contact]"
4. Claude AI analyzes and decides:
- Which escalation method has highest success rate for this user
 - Optimal timing (immediate or wait 30 sec)
 - Message content (personalized based on user preferences)
 - Fallback if primary method fails
5. System returns decision with confidence score

Phase 3: Execute Escalation

6. Based on AI decision, system selects escalation method:

If Call Selected: 7a. System initiates phone call via Twilio 7b. System plays pre-recorded or TTS message: "This is your RiseUp alarm. Please confirm you are awake by pressing 1." 7c. User presses 1 to confirm 7d. System records confirmation and stops escalation 7e. Proceed to step 10

If SMS Selected: 7b. System sends SMS via Twilio: "RiseUp: Your alarm has triggered. Reply AWAKE to confirm." 7c. User replies with "AWAKE" or similar 7d. System detects response and records confirmation 7e. Proceed to step 10

If Alexa Selected: 7c. System sends request to Alexa API 7d. Alexa announces: "Your RiseUp alarm has triggered. Say yes to confirm you're awake." 7e. User says "yes" or similar 7f. System detects voice response and records confirmation 7g. Proceed to step 10

If Emergency Contact Selected: 7h. System calls/texts emergency contact: "RiseUp alert: [User name] is not responding to alarm. Please check on them." 7i. Emergency contact acknowledges 7j. Proceed to step 10

Phase 4: Completion

8. System logs escalation event:
- Method used
 - Time of escalation
 - User response time

- Outcome (successful/failed)
- 9. System updates user profile with escalation effectiveness data
- 10. System notifies user (if they respond) of escalation taken
- 11. Escalation ends

Alternative Flows

A1: Primary Escalation Method Fails

- System attempts call via Twilio
- Call connection fails (user phone off, unreachable)
- System waits 30 seconds
- System attempts fallback method (SMS instead)
- User responds to SMS
- Continue from step 8 in main flow

A2: Multiple Escalation Attempts Needed

- First escalation attempt (call) → No response
- System waits 1 minute
- Second escalation attempt (SMS) → No response
- System waits 2 minutes
- Third attempt (Alexa alert) → User responds
- System logs all attempts and outcomes
- Continue from step 8 in main flow

A3: User Disables Escalation

- User has disabled all escalation methods in settings
- Alarm triggers but times out without verification
- System displays on user's dashboard: "Alarm missed at 7:00 AM"
- No escalation occurs
- System logs missed alarm for user review

A4: Multiple Alarms Trigger Simultaneously

- User has two alarms set 5 minutes apart
- Both trigger within short window
- System prioritizes based on alarm type/importance
- Escalates the higher-priority alarm first
- Delays second escalation by 30-60 seconds

Exception Flows

E1: LLM Decision Fails

- System sends request to Claude API
- API timeout or failure occurs
- System falls back to rule-based escalation:
 - Use method that worked last time
 - If no history, default to: call → SMS → Alexa

- Continue with Phase 3

E2: All Escalation Methods Unavailable

- User has disabled all escalation methods
- Call, SMS, Alexa all disabled
- User has no emergency contact
- System logs escalation failure
- System displays alert on user's dashboard: "Alarm at 7:00 AM went unanswered"
- User receives in-app notification on next login

E3: User Rejects Escalation

- System calls user
- User explicitly says "do not call again" or ignores multiple attempts
- System detects rejection pattern
- System updates user's escalation settings
- System respects user preferences for future alarms

E4: Emergency Contact Unreachable

- Emergency contact selected as escalation method
- Contact's phone is off or unavailable
- System attempts 3 times with 30-sec delays
- On final failure, system falls back to SMS or logs failure
- System notifies user of failed emergency contact

Postconditions

- Escalation method has been attempted (at least once)
- Escalation outcome is recorded in database
- User profile updated with escalation effectiveness data
- AI learning engine updates escalation strategy for future use
- If successful: User has responded and confirmed wakefulness
- If failed: User dashboard shows missed alarm with escalation attempts

Notes

- AI learns which escalation methods work best for each user
- Escalation decisions improve over time with historical data
- System respects user privacy (no escalation if all methods disabled)
- Emergency contact is last resort, used rarely
- Escalation logs help identify users who consistently miss alarms
- System can suggest alarm time/verification task changes to user

UC-04: User Creates a Task Reminder

Use Case ID

UC-04

Use Case Name

Create Task Reminder

Related Feature(s)

F07 — Natural-Language Reminder Creation; F08 — Reminder Management

Primary Actor

User

Secondary Actors

- System (Task Scheduler)
- LLM Service (default OpenAI gpt-4o for NL parse; Claude optional)

Goal

User creates a task reminder and the system schedules it to notify the user at the appropriate time.

Preconditions

- User is logged into RiseUp
- User has access to web app or voice interface
- System has current time and user's timezone

Trigger

User clicks "New Reminder" or "Add Task" or speaks voice command to system.

Main Success Scenario

Via Web App:

1. User clicks "New Reminder" button on dashboard
2. System displays task creation form with fields:
 - Task title/description
 - Due date
 - Due time
 - Priority (low/medium/high/critical)
 - Category (work, personal, health, other)
 - Notification type (push, SMS, call, email)
3. User enters task description (e.g., "Call mom")
4. User sets due date (today, tomorrow, specific date)
5. User sets due time (HH:MM)
6. User selects priority level
7. User optionally selects category
8. User optionally adds notes or description
9. User clicks "Save Reminder"
10. System validates task data

11. System schedules reminder notification at due time
12. System displays confirmation: "Reminder set for [date] at [time]"
13. Reminder appears on dashboard in task list
14. Task is stored in database

Via Voice:

1. User says "Hey RiseUp, remind me to [task] at [time]"
2. System captures voice input
3. LLM parses natural language:
 - Extracts task: "call mom"
 - Extracts time: "tomorrow at 9am"
 - Infers priority: medium (default)
 - Infers category: personal
4. System confirms: "Reminder set to call mom tomorrow at 9 AM. Correct?"
5. User confirms "yes"
6. Continue from step 10 in main flow

Alternative Flows

A1: User Creates Recurring Task

- At step 5, user sets repeat pattern: daily, weekly, weekdays, custom
- System schedules multiple reminders (one per occurrence)
- User can edit or delete recurring series
- Each occurrence is logged separately for completion tracking

A2: User Sets Multiple Reminders for Same Task

- User sets primary reminder at 9 AM
- User clicks "Add another reminder time"
- User sets secondary reminder at 9:30 AM (escalation reminder)
- System creates two separate notifications
- Both reminders appear on dashboard

A3: User Cancels Task Creation

- User fills form but clicks "Cancel"
- No reminder is saved
- User returns to dashboard
- No data loss

A4: User Edits Existing Task Reminder

- User clicks "Edit" on existing reminder
- System loads task details into form
- User modifies due time, priority, category
- User clicks "Save"
- System updates database
- Notification schedule is recalculated if time changed

A5: User Marks Task as Complete

- Task notification is sent
- User clicks "Complete" button or says "done"
- System marks task as completed
- Task moves to completed list
- Completion time is recorded for analytics

Exception Flows

E1: Invalid Task Title

- User leaves task title blank
- User clicks "Save Reminder"
- System displays error: "Task title is required"
- User must enter task title
- Continue from step 9 in main flow

E2: Invalid Time Entry

- User enters invalid time format
- System displays error: "Please enter time in HH:MM format"
- User corrects entry
- Continue from step 9 in main flow

E3: Past Time/Date Entry

- User sets reminder for 8 AM today, but current time is 9 AM
- System detects past time
- System offers: "This time has passed. Set for tomorrow instead?"
- User confirms or sets new time
- Continue from step 10 in main flow

E4: Voice Parse Fails

- User says reminder with ambiguous wording
- LLM cannot extract clear task or time
- System asks for clarification: "Did you say remind you to [X]? At what time?"
- User provides missing information
- LLM re-parses
- If successful, continue from step 10 in main flow

Postconditions

- Task reminder is stored in database
- Notification is scheduled at specified time
- Task appears on user's dashboard/task list
- User receives confirmation
- Completion tracking is enabled

Notes

- Reminders can be set via app or voice
 - AI parses natural language for convenience
 - Users can set recurring reminders
 - Completion data helps system learn user patterns
 - Failed/missed reminders are logged for analytics
-

UC-05: System Sends Prioritized Task Reminders

Use Case ID

UC-05

Use Case Name

Send Prioritized Task Reminders

Related Feature(s)

F09 — AI Reminder Prioritization; F10 — Reminder Escalation Levels

Primary Actor

System (Task Scheduler & AI Prioritizer)

Secondary Actors

- User (recipient)
- LLM Service (Claude / OpenAI)
- Firebase / Twilio (notifications)

Goal

At scheduled time, system intelligently prioritizes and sends task reminders to maximize user engagement.

Preconditions

- Task reminders have been created (UC-04)
- Current time matches or approaches scheduled reminder time
- User is available to receive notification
- System has user's notification preferences configured

Trigger

System's scheduler detects that reminder notification time has arrived.

Main Success Scenario

Phase 1: Reminder Trigger Check

1. System checks if multiple reminders are due simultaneously
2. If only one reminder:
 - Go to step 5 (single reminder flow)
3. If multiple reminders at same time:
 - Continue to step 4 (prioritization flow)

Phase 2: AI Prioritization (if multiple)

4. System sends request to Claude LLM:
 - "User has 3 reminders due now: [list]"
 - "User priority levels: [task1]=high, [task2]=medium, [task3]=low"
 - "User's typical task completion rates: work=80%, personal=50%, health=90% "
 - "Time of day: [morning/afternoon/evening]"
 - "Recent task completion history: [data]"
 - Decision: "Which task should be prioritized?"
5. Claude analyzes and recommends priority order:
 - Health task (highest completion rate, high priority)
 - Work task (medium priority, work hours)
 - Personal task (low priority, low completion rate)
6. System orders reminders by AI recommendation

Phase 3: Send Primary Reminder

7. System determines notification method based on user preference:
 - Push notification (default)
 - SMS
 - Email
 - Combination
8. System sends primary reminder:
 - Content: "[Task name]: [Description]"
 - Example: "Call Mom: Don't forget to call your mom"
 - Includes completion button/action
9. System starts timer (track user response time)
10. User receives reminder notification
11. If user completes task immediately → Log completion, end flow
12. If user doesn't respond within 5 minutes → Go to step 13 (escalation reminder)

Phase 4: Escalation Reminder (if needed)

13. System sends secondary "escalation" reminder:
 - Slightly different tone/message
 - Example: "Hey! Still need to call Mom. Tap to complete."
 - Sent 5 minutes after primary reminder
14. If user completes task → Log completion, end flow
15. If user still doesn't respond within 10 more minutes → Go to step 16 (final reminder)

Phase 5: Final Reminder

16. System sends final reminder:
 - More urgent tone
 - Example: "Last reminder: Call Mom (high priority). Complete now?"
 - Sent 15 minutes after primary reminder
17. If user completes → Log completion, end flow
18. If user doesn't complete → Log as missed/incomplete, end flow

Phase 6: Logging & Learning

19. System records:
 - Task name
 - Scheduled time vs. actual completion time
 - Number of reminders needed before completion
 - Notification method used
 - User response time
 - Task category and priority
20. System updates user profile:
 - Task completion patterns (time of day, category, priority)
 - Optimal reminder frequency for this user
 - Which notification methods are most effective
21. Data feeds into future prioritization logic

Alternative Flows

A1: User Completes Task Immediately

- User receives primary reminder
- User clicks "Complete" button or marks done
- System logs completion at reminder time (or close to it)
- No escalation reminders sent
- Data recorded for analytics
- Continue from step 19

A2: User Snoozes Reminder

- User receives reminder
- User clicks "Snooze 15 min"
- Reminder is rescheduled for 15 min later
- At new time, primary reminder is resent
- Continue from step 9 (restart timer for new time)

A3: User Marks Task as "Done Earlier"

- User receives reminder
- But task was already completed earlier in the day
- User clicks "Already done" option
- System removes reminder and logs completion
- System adjusts reminder scheduling for similar tasks

A4: Single Reminder with High Priority

- Only one high-priority reminder is due
- System sends immediately with urgent notification style
- No AI prioritization needed (only one task)
- Continue from step 7

A5: Reminder for Recurring Task

- Recurring task (e.g., "Drink water") set to send multiple times/day
- Each occurrence is sent at scheduled time
- User can mark individual occurrences as complete
- Series can be completed or skipped
- Data tracked per occurrence

Exception Flows

E1: User Disabled Notifications

- Reminder is due
- User has notifications disabled in settings
- System checks last login time
- If recent: Schedule reminder for next login
- If old: Log missed reminder for user to review later

E2: User's Device Offline

- Reminder is due
- System attempts to send push notification
- Delivery fails (device offline)
- System queues notification
- When device comes online, queued notification is delivered
- Completion is still tracked if user completes within reasonable time

E3: LLM Prioritization Fails

- System sends request to Claude for prioritization
- API timeout or error occurs
- System falls back to simple prioritization:
 - Sort by: priority level → category → due time
 - Send highest priority first
- Continue with Phase 3

E4: User Dismisses All Reminders

- User receives 3 escalation reminders
- User dismisses each without completing task
- System logs as "missed/incomplete"
- On dashboard, task appears as "incomplete" with red flag
- AI learns that this task/category has low completion rate
- System may suggest: "You often miss [category] tasks. Change reminder time?"

Postconditions

- Reminder notification has been sent to user
- Completion status is recorded (completed/incomplete/missed)
- Response time is logged
- User behavior data is stored for learning engine
- Task appears on user's dashboard with completion status
- AI prioritization logic is updated with new data

Notes

- Multiple reminders prioritized by AI (not just sequential)
 - Escalation reminders increase likelihood of completion
 - System learns optimal reminder frequency per user
 - Missed reminders help identify problematic tasks or times
 - Completion data drives future task scheduling recommendations
 - Privacy: All data is tied to user's own profile, not shared
-
-

UC-06: Manage Existing Reminders

Use Case ID

UC-06

Use Case Name

Manage Existing Reminders

Related Feature(s)

F08 — Reminder Management

Primary Actor

User

Secondary Actors

- System (ReminderScheduler)

Goal

Edit, snooze, complete, or delete existing reminders (including recurring series choices).

Preconditions

- At least one reminder exists (UC-04)

Trigger

User opens task dashboard / runs `riseup task list|edit|done|snooze|delete`

Main Success Scenario

1. User lists reminders (GUI filters or CLI list)
2. User selects edit / complete / snooze / delete
3. System validates and updates `Reminder` + schedule
4. Observers refresh dashboard / logs
5. Confirmation shown

Alternative/Exception Flows

- Recurring edit: ask this occurrence vs entire series
- Already completed: reject further snooze
- DB failure: retry message

Postconditions

- Reminder state and schedule match user intent

UC-07: View Analytics & Recommendations

Use Case ID

UC-07

Use Case Name

View Analytics and AI Recommendations

Related Feature(s)

F11 — Behavioral Learning; F12 — Analytics Dashboard; F13 — AI Recommendations

Primary Actor

User

Secondary Actors

- System (LearningEngine, AnalyticsEngine)
- LLM Service (for F13)

Goal

Inspect sleep/completion stats and optionally accept AI setting suggestions.

Preconditions

- User has some history (or empty-state messaging if not)

Trigger

User opens Insights / Recommendations, or `riseup stats` / `riseup recommend`

Main Success Scenario

1. User selects date range
2. System returns charts/tables from `AnalyticsEngine`
3. User requests recommendations
4. `RecommendationService` + LLM produce validated suggestions
5. User accepts one → `UserPreferences` updated

Alternative/Exception Flows

- Insufficient data: no suggestions, explain why
- LLM failure: show rule-based tips only
- Invalid suggestion (e.g. disables all escalation): dropped

Postconditions

- Stats viewed; accepted recommendations applied and logged

UC-08: Manage Escalation Preferences & Emergency Contact

Use Case ID

UC-08

Use Case Name

Manage Escalation Preferences and Emergency Contact

Related Feature(s)

F06 — Emergency Contact & Escalation Preferences

Primary Actor

User

Secondary Actors

- Emergency contact (consent recipient)
- Twilio (test SMS/call)

Goal

Configure enabled channels, quiet hours, and consenting emergency contact.

Preconditions

- User authenticated

Trigger

Settings → Escalation, or `riseup prefs` / `riseup contact`

Main Success Scenario

1. User toggles channels and quiet hours
2. User adds emergency contact
3. System sends consent request; contact confirms
4. Optional test message via ToolManager
5. Preferences saved

Alternative/Exception Flows

- Invalid phone: error
- Consent not granted: contact excluded from F05
- All channels off: warn that alarms will not escalate

Postconditions

- `UserPreferences` / `EmergencyContact` persisted

UC-09: View History & Decision Explanation

Use Case ID

UC-09

Use Case Name

View History and Decision Explanation

Related Feature(s)

F14 — History & Decision Explanation

Primary Actor

User

Secondary Actors

- System (HistoryController)

Goal

Browse past alarms/escalations/reminders and open “Why?” for AI decisions.

Preconditions

- Events exist in logs

Trigger

History page, or `riseup history` / `riseup escalation explain <id>`

Main Success Scenario

1. User filters by type/date
2. System lists events
3. User opens an AI decision
4. System shows reasoning, confidence, inputs, fallback flag

Alternative/Exception Flows

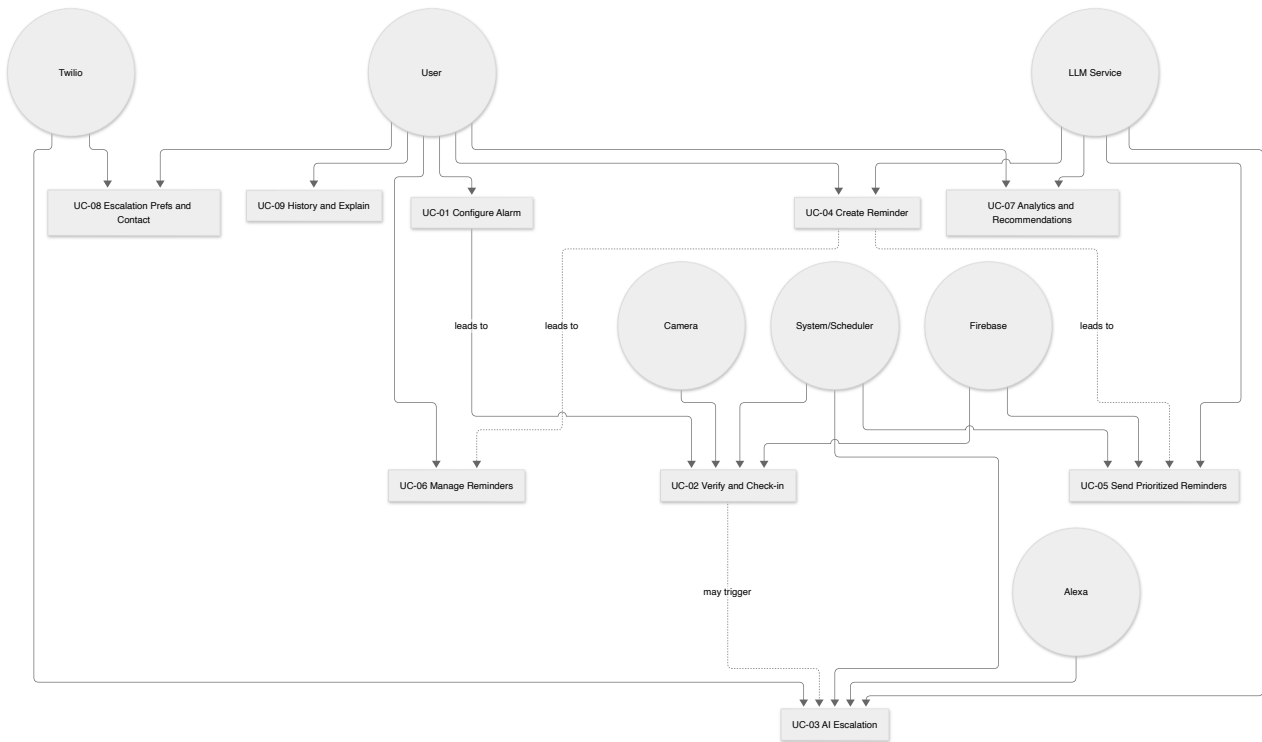
- Not found: 404 message
- Rule-based fallback decision: labelled clearly

Postconditions

- User understands why an action was taken (explainability constraint)

Use Case Diagram

Actors: **User**, **System/Scheduler**, **LLM Service**, **Twilio**, **Alexa**, **Camera**, **Firebase**.



Note: There is **no** UC-03 → UC-05 dependency. Escalation and reminder prioritization are separate flows that both may use the LLM.

PNG: [uml-diagrams/UseCaseDiagram.png](#) .

Use Case Dependencies

- **UC-01** prerequisite for **UC-02**
- **UC-02** may trigger **UC-03** (timeout / missed check-in)
- **UC-04** prerequisite for **UC-05** and enables **UC-06**
- **UC-08** configures channels used by **UC-03**
- **UC-07** / **UC-09** read data produced by alarms, escalations, and reminders
- **UC-03** and **UC-05** both may call the LLM; they do **not** call each other

Summary Table

Use Case	Actor	Related Features	Trigger	Key decision
UC-01	User	F01	New Alarm	Task type, schedule
UC-02	User/System	F02, F03	Alarm time	Verify / check-in
UC-03	System	F04, F05	Timeout	Agent chooses channel
UC-04	User	F07, F08	New reminder / NL	Parse & confirm
UC-05	System	F09, F10	Due time	Prioritize & nudge
UC-06	User	F08	Edit/done/snooze	Series vs one
UC-07	User	F11–F13	Insights	Accept recommendation

Use Case	Actor	Related Features	Trigger	Key decision
UC-08	User	F06	Settings	Consent & channels
UC-09	User	F14	History	Explain AI decision

RiseUp - Design Decisions & Justifications

Chapter 4 — source: docs/stage-1-design/design-decisions.md

Chapter overview. Chapter 4 records **16 design decisions**. Each states the decision, why it was made, the alternatives considered and the option chosen:

- **Architecture (AD-01 – AD-05):** layered client-server; Strategy for verification tasks; a separate EscalationEngine; LLMClient + LLMProvider; the AgentController multi-step loop.
- **AI integration (AI-01 – AI-03):** dual LLM providers; AI for decisions but not execution; structured JSON prompts.
- **Data (DP-01 – DP-03):** PostgreSQL; Redis; pre-aggregated UserProfile metrics.
- **Testing & reliability (TR-01 – TR-02):** unit tests plus KUMA behavioural tests; fallbacks everywhere.
- **UX (UX-01 – UX-03):** fallback escalation ladder; delayed check-in; escalating reminder levels.

The chapter closes with the SOLID and other design principles applied, future enhancements and a conclusion. [Chapter 8 — Decision index](#) lists all decisions in one table.

This document explains the important architectural and design decisions made during Stage 1, why they were chosen, and what alternatives were considered.

Table of Contents

1. [Architectural Decisions](#)
2. [AI Integration Decisions](#)
3. [Data & Persistence Decisions](#)
4. [Testing & Reliability Decisions](#)
5. [User Experience Decisions](#)

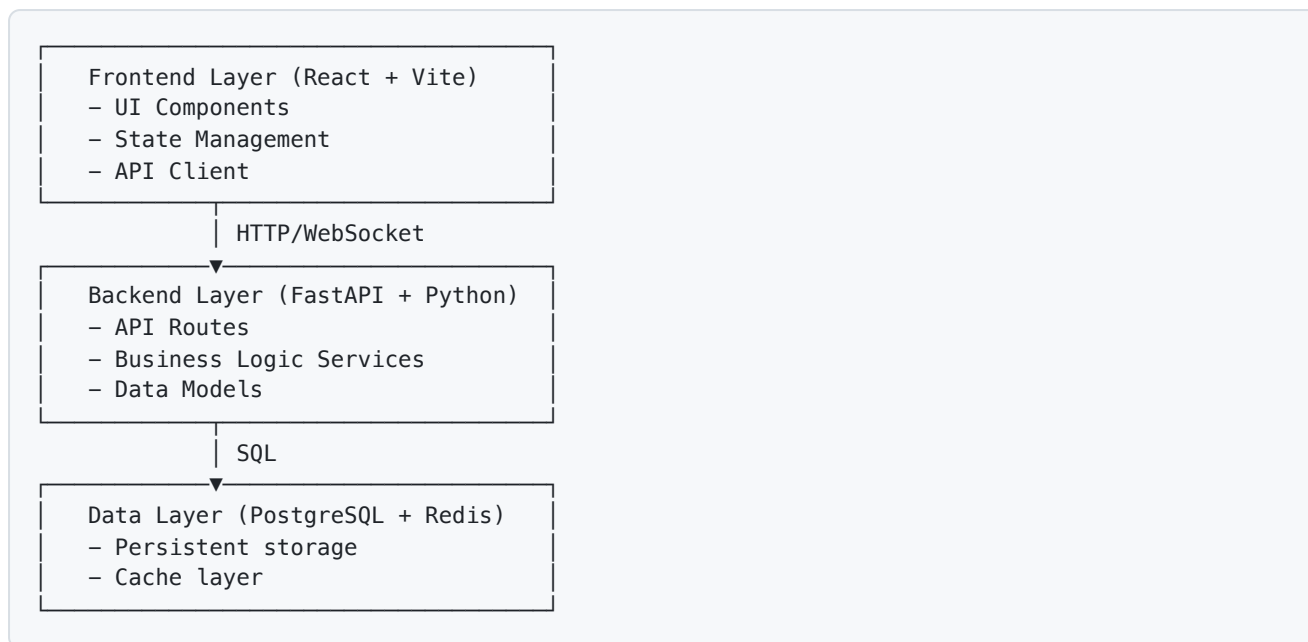
Design **patterns** (Strategy, Adapter, Command, Observer, Factory Method) are documented in [design-patterns.md](#), not duplicated here.

Architectural Decisions

AD-01: Layered Architecture (Client-Server)

Decision: Separate frontend (React) and backend (FastAPI) with REST API + WebSocket communication.

Architecture:



Why This Design:

- **Separation of Concerns:** UI logic separate from business logic
- **Scalability:** Backend can be scaled independently from frontend
- **Cross-Platform:** Same backend serves web, mobile (future), and CLI
- **Testability:** Each layer can be tested independently
- **Maintainability:** Clear responsibilities for each layer

Alternatives Considered:

1. Monolithic (Full-stack in FastAPI):

- ❌ Harder to maintain (mixing templates with business logic)
- ❌ Difficult to scale frontend separately
- ❌ Cannot easily add mobile app later

2. Microservices (separate services for alarms, escalation, reminders):

- ❌ Too complex for current scope
- ❌ Would require message queue infrastructure
- ❌ Operational overhead (multiple deployments)

Chosen: Layered (best balance of simplicity, maintainability, scalability)

AD-02: Strategy Pattern for Verification Tasks

Decision: Abstract `VerificationTask` class with concrete implementations: `ObjectDetectionTask`, `MathTask`, `TriviaTask`, `CustomTask`.

Class Hierarchy:

```
VerificationTask (abstract)
├── ObjectDetectionTask
├── MathTask
├── TriviaTask
└── CustomTask
```

Why This Design:

- **Open/Closed Principle:** Easy to add new task types without modifying existing code
- **Polymorphism:** Different tasks have different validation logic, but same interface
- **Flexibility:** Users can choose task types, system can add types later
- **Testing:** Each task type can be tested independently

Alternatives Considered:

1. String-based task type with if/else:

```
if task_type == "photo":
    # photo logic
elif task_type == "math":
    # math logic
```

- ❌ Violates DRY (logic scattered)
- ❌ Hard to add new types (must modify existing code)
- ❌ Cannot use polymorphism
- ❌ Testing is more complex

2. Single `VerificationTask` class with mode parameter:

- ❌ God Object (too many responsibilities)
- ❌ Hard to maintain (all logic in one class)

Chosen: Strategy Pattern (clean, extensible, testable)

AD-03: Separate Escalation Engine Service

Decision: Create dedicated `EscalationEngine` class that orchestrates escalation decisions.

EscalationEngine Responsibilities:

- Gather user context from `UserProfile` and `EscalationLog`
- Call LLM for decision-making
- Coordinate with `ToolManager` to execute decision
- Record outcome in `EscalationLog`
- Update `UserProfile` with learning data

Why This Design:

- **Single Responsibility:** Escalation logic is isolated
- **Reusability:** Can be called from multiple places (timeout, check-in failure, etc.)
- **Testing:** Can test escalation logic independently

- **Evolution:** Can improve escalation algorithm without affecting alarm system
- **Learning:** Clear pipeline for data collection → AI → action

Alternatives Considered:

1. Inline escalation in Alarm class:

- ❌ Alarm class becomes too large
- ❌ Escalation tied to alarm timing
- ❌ Cannot reuse for check-in failures
- ❌ Hard to test

2. Escalation logic inside LLMClient:

- ❌ Mixes AI integration with orchestration
- ❌ LLM shouldn't know about Twilio/Alexa
- ❌ Cannot test without calling LLM

Chosen: Separate Service (clean separation, testable, reusable)

AD-04: LLMClient + LLMProvider (Adapter + Strategy)

Decision: Domain code talks only to `LLMClient`. `LLMClient` delegates text generation to an `LLMProvider` strategy (`ClaudeProvider` or `OpenAIProvider`). Escalation **decisions** live on

`EscalationEngine.decide_escalation()` (and the agent loop on `AgentController`), not on `LLMClient`.

Models (Stage 1 design):

- Anthropic: `claude-sonnet-4-5` (default for escalation / recommendations)
- OpenAI: `gpt-4o` (default for NL reminder parsing; interchangeable via config)

Responsibilities:

- `LLMProvider.complete(prompt) -> str` — vendor call only
- `LLMClient.send_request()` — retries, timeouts, provider selection
- `PromptBuilder` / `ResponseParser` — prompt construction and JSON validation
- `EscalationEngine.decide_escalation()` — builds context, calls LLM, validates method

Example Usage:

```
# BAD: decision logic and vendor SDK mixed
response = anthropic.messages.create(model="claude-sonnet-4-5", messages=[...])

# GOOD: engine decides; client only talks to a provider
decision = escalation_engine.decide_escalation(alarm_id) # may call
llm_client.send_request()
```

Alternatives Considered:

1. **Direct API calls everywhere:** scattered keys, untestable, vendor lock-in
2. `LLMClient.decide_escalation()` : mixes orchestration with transport (rejected)

Chosen: `LLMClient` + `LLMProvider` strategies; decisions stay in domain/agent layer

AD-05: AgentController multi-step loop (not a one-shot LLM call)

Decision: AI features that act on the world (escalation, NL reminder confirmation side-effects, recommendation apply) go through `AgentController.run(goal, context)`:

1. **Observe** — load `UserProfile` + recent `EscalationLog` / reminder logs (agent memory)
2. **Decide** — LLM returns a structured action (method, tool, args, reasoning)
3. **Act** — `ToolManager` executes via `Tool` adapters
4. **Observe outcome** — success/failure recorded
5. **Re-decide** if the tool failed and a fallback remains; else stop with deterministic fallback ladder

This satisfies the Stage 1 rule that a simple “prompt → display text” chatbot is not enough.

Memory:

- Short-term: current alarm/reminder context + last tool result
 - Long-term: `UserProfile` aggregates + `EscalationLog` history
-

AI Integration Decisions

AI-01: Dual LLM providers (Claude + OpenAI)

Decision: Use **both** Anthropic Claude (`claude-sonnet-4-5`) and OpenAI (`gpt-4o`) behind `LLMProvider` . Default routing: Claude for escalation/recommendations; OpenAI for NL parsing. Either provider can be selected via config for any AI feature.

Use the LLM agent for:

- Escalation method selection (UC-03)
- Task reminder prioritization (UC-05)
- User behavior analysis
- Natural language command parsing

Why Claude:

- **Reasoning:** Excels at multi-step reasoning (analyze history → decide best approach)
- **Reliability:** Strong instruction-following (can follow escalation rules)
- **Context:** Can process complex user history and make contextual decisions
- **Cost:** Reasonable API pricing for this use case
- **Academic:** Available for educational projects
- **Interpretability:** Decisions can be explained to users

Capabilities Used:

Escalation Decision:

```
Context: {
  user: "John",
  last_escalation: "call",
  call_success_rate: 0.95,
  sms_success_rate: 0.4,
  available_methods: ["call", "sms", "alexa"],
  current_time: "7:30 AM",
  user_timezone: "EST"
}
```

Prompt: "Given this user context, which escalation method is best?"

Response: "CALL (95% success rate, morning time suggests user responsive to calls)"

Alternatives Considered:

1. Rule-based heuristics (if/else):

```
if call_success_rate > 0.8:
    use_call()
elif sms_success_rate > 0.6:
    use_sms()
```

- ❌ Brittle (doesn't handle edge cases)
- ❌ Hard to maintain (many rules to manage)
- ❌ Cannot adapt to new situations
- ❌ No learning capability

2. Machine Learning model (train on historical data):

- ❌ Requires large historical dataset (cold-start problem)
- ❌ Infrastructure complexity (model training, deployment)
- ❌ Maintenance burden (model monitoring, retraining)
- ⚠️ Possible future enhancement after sufficient data collected

3. Simple random selection:

- ❌ No intelligence
- ❌ Poor user experience
- ❌ Cannot learn

Chosen: Dual providers (`claude-sonnet-4-5` + `gpt-4o`) behind `LLMProvider` , driven by `AgentController`

AI-02: AI for Prioritization (Not Execution)

Decision: Use AI ONLY for decision-making, not for executing deterministic tasks.

AI Used For:

- ✅ Deciding escalation method
- ✅ Prioritizing reminders
- ✅ Analyzing user patterns

-  Generating insights

AI NOT Used For:

-  Verifying photo (use computer vision library)
-  Validating math answer (deterministic calculation)
-  Scheduling alarms (use APScheduler)
-  Storing data (use PostgreSQL)

Why This Design:

- **Reliability:** Deterministic tasks must work 100% reliably (don't trust LLM for this)
- **Cost:** Deterministic tasks are cheaper with libraries
- **Speed:** Deterministic tasks are faster (no API calls)
- **Testing:** Deterministic tasks are easier to test

Example:

```
# CORRECT: domain/agent decides; tools execute
decision = escalation_engine.decide_escalation(alarm_id) # AI orchestrated
success = tool_manager.execute_escalation(decision.method) # Deterministic tools

# WRONG: AI for everything
response = llm_client.verify_photo(photo_data) # Don't do this!
```

Alternatives Considered:

1. **Use LLM for all decisions (including verification):**
 -  Expensive (API calls for every verification)
 -  Slow (latency)
 -  Unreliable (hallucinations possible)

Chosen: AI for decisions, deterministic for execution

AI-03: Prompt Engineering for Consistency

Decision: Carefully design prompts to ensure consistent, reliable LLM responses.

Escalation Prompt Structure:

CONTEXT:

- User: [name]
- Escalation history: [past methods and outcomes]
- Success rates: [method → success rate]
- Current conditions: [time, device status, etc.]
- Available methods: [call, SMS, Alexa, emergency contact]

TASK:

Analyze the user history and recommend ONE escalation method.

OUTPUT FORMAT:

```
{
  "method": "call" | "sms" | "alexa" | "emergency_contact",
  "reasoning": "[explain why this method]",
  "confidence": 0.0-1.0,
  "fallback": "[alternate method if first fails]"
}
```

CONSTRAINTS:

- ONLY use methods in the available list
- MUST choose the method with highest historical success rate
- DO NOT make up methods

Why This Design:

- **Consistency:** Structured format ensures predictable output
- **Fallback:** Always has a backup plan
- **Reasoning:** Explains decision (for debugging/transparency)
- **Confidence:** Indicates how certain the LLM is
- **Constraints:** Prevents LLM from making invalid recommendations

Alternatives Considered:

1. **Unstructured prompts:**

-  Unpredictable output format
-  Hard to parse programmatically
-  Easy for LLM to make invalid suggestions

Chosen: Structured prompts with JSON output

Data & Persistence Decisions

DP-01: PostgreSQL for Persistent Storage

Decision: Use PostgreSQL for all persistent data (alarms, reminders, users, logs).

Data Stored:

- User accounts and preferences
- Alarms and reminders
- Verification task history
- Escalation logs
- Check-in records

- Behavioral analytics data

Why PostgreSQL:

- **ACID:** Guarantees data consistency (important for alarms!)
- **Reliability:** Won't lose data if app crashes
- **Querying:** Can analyze historical data (behavior patterns)
- **Relationships:** Can model complex relationships (user → alarms → check-ins)
- **Scalability:** Handles large datasets well

Schema Example:

```
-- Users
CREATE TABLE users (
  id UUID PRIMARY KEY,
  email VARCHAR UNIQUE,
  timezone VARCHAR,
  created_at TIMESTAMP
);

-- Alarms
CREATE TABLE alarms (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users,
  time TIME,
  sound VARCHAR,
  verification_task_type VARCHAR,
  repeat_pattern VARCHAR,
  is_active BOOLEAN,
  created_at TIMESTAMP
);

-- Escalation Logs (for learning)
CREATE TABLE escalation_logs (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users,
  alarm_id UUID REFERENCES alarms,
  method VARCHAR,
  timestamp TIMESTAMP,
  response_status VARCHAR,
  response_time_seconds INT
);
```

Alternatives Considered:

1. SQLite (file-based):

- ❌ Not suitable for multi-user app
- ❌ Limited concurrency

2. NoSQL (MongoDB):

- ❌ Overkill for structured data
- ❌ Harder to analyze historical data
- ❌ No ACID guarantees by default

Chosen: PostgreSQL (reliable, queryable, ACID)

DP-02: Redis for Caching & Real-Time

Decision: Use Redis for:

- Caching frequently accessed data (user profiles, escalation success rates)
- Session management
- Real-time WebSocket message queue
- Celery task queue (for background jobs)

Why Redis:

- **Speed:** In-memory → nanosecond latency
- **Real-time:** Perfect for WebSocket-based notifications
- **Pub/Sub:** Built-in message broadcasting
- **Queue:** Celery integration for background tasks
- **Cache:** Reduces database load

Usage Patterns:

```
# Cache user profile (update less frequently)
redis.set(f"user_profile:{user_id}", user_profile_json, expire=1800)

# Cache escalation success rates (update hourly)
redis.set(f"escalation_stats:{user_id}", stats_json, expire=3600)

# Queue background escalation task
celery_task.delay(alarm_id=123, user_id=456)

# WebSocket broadcast (send to all connected users)
redis.publish("alarm_triggered", json.dumps(alarm_event))
```

Alternatives Considered:

1. **Database cache (no Redis):**

- **✗** Slower (must query database)
- **✗** Cannot do real-time notifications easily

Chosen: Redis + PostgreSQL (fast caching, reliable persistence)

DP-03: UserProfile for Aggregated Metrics

Decision: Store aggregated behavioral metrics in `UserProfile` rather than computing from raw logs every time.

UserProfile Stores:

```

class UserProfile:
    user_id: UUID

    # Escalation effectiveness
    escalation_stats: {
        "call": {"success_rate": 0.95, "avg_response_time": 120},
        "sms": {"success_rate": 0.40, "avg_response_time": 300},
        "alexa": {"success_rate": 0.60, "avg_response_time": 180}
    }

    # Alarm patterns
    alarm_patterns: {
        "avg_wake_time": "07:30",
        "success_rate_by_hour": {
            "06": 0.8, "07": 0.95, "08": 0.92, ...
        }
    }

    # Task completion
    task_completion_rates: {
        "work": 0.85,
        "personal": 0.60,
        "health": 0.95
    }

    # Recent updates
    updated_at: DateTime

```

Why This Design:

- **Performance:** Quick lookups (no computation needed)
- **AI Context:** LLM gets precomputed stats, not raw logs
- **Reduced LLM Cost:** Smaller context → smaller API requests
- **Real-time:** Stats always available
- **Versioning:** Can track how stats change over time

Update Strategy:

```

Events happen throughout the day
↓
Logged in EscalationLog, ReminderLog, etc.
↓
Nightly: LearningEngine computes new stats
↓
Update UserProfile with new metrics
↓
Next escalation/prioritization uses updated data

```

Alternatives Considered:

1. **Compute stats on-the-fly:**
 - **✗** Slow (must aggregate logs every time)
 - **✗** Expensive (complex queries)
 - **✗** LLM gets delayed insights

Chosen: Pre-aggregated metrics in UserProfile

Testing & Reliability Decisions

TR-01: Separate Test Strategies for Deterministic vs. AI Components

Decision:

- **Deterministic Components:** Traditional unit tests (pytest)
- **AI/LLM Components:** Behavioral tests (KUMA framework)

Deterministic Components (Unit Tests):

```
# Alarm scheduling
def test_alarm_schedule():
    alarm = Alarm(time="07:00", user_id=123)
    job = alarm_scheduler.schedule(alarm)
    assert job.trigger_time == "07:00"

# Math verification
def test_math_task_validation():
    task = MathTask(equation="2+2")
    assert task.validate_answer(4) == True
    assert task.validate_answer(5) == False

# Escalation logging
def test_escalation_logged():
    escalate(user_id=123, method="call")
    log = database.get_escalation_log(user_id=123)
    assert log.method == "call"
```

AI/LLM Components (Behavioral Tests):

```
Test: BR-01 Enabled Channel Selection
- Input: Alarm timeout; user prefs enable only SMS and Alexa (call disabled)
- Expected: Agent chooses only from enabled channels (SMS or Alexa)
- Prohibited: Selecting call or emergency contact when disabled / no consent
- Pass Criteria: Decision method ∈ enabled set; invalid LLM pick rejected

Test: BR-02 LLM Invalid JSON Fallback
- Input: Escalation needed; LLM returns malformed / non-JSON decision
- Expected: One retry, then EscalationEngine.fallback_ladder(); decision logged with fallback flag
- Prohibited: Silent hang or executing an unvalidated method string
- Pass Criteria: Fallback method used; EscalationLog marks fallback_used=true

Test: BR-03 Escalation Adapts to User History
- Input: UserProfile shows call success 95%, SMS 40%
- Expected: Agent prefers CALL when call is enabled
- Evaluation: Decision matches history-weighted preference (or documented fallback)
```

Why This Design:

- **Reliability:** Deterministic logic must work 100%
- **Pragmatism:** Cannot demand 100% from LLM
- **Different Metrics:** Success = different things

- **Cost:** Cheaper to unit test deterministic code
- **Speed:** Unit tests run fast (no API calls)

Alternatives Considered:

1. Only unit tests (no AI testing):

- ✗ Cannot verify LLM behaves correctly
- ✗ Cannot catch AI failures in production

2. All behavioral tests (expensive, slow):

- ✗ Expensive (API calls for every test)
- ✗ Slow feedback loop
- ✗ Cannot test deterministic logic easily

Chosen: Dual strategy (unit tests + behavioral tests)

TR-02: Error Handling & Fallbacks

Decision: Every external API call has error handling and fallback logic.

Escalation Fallback Chain:

```
Try: Call user via Twilio
  ↓ (Fails: network error)
Try: Send SMS via Twilio
  ↓ (Fails: user has no phone)
Try: Alexa alert
  ↓ (Fails: no Alexa device)
Try: Contact emergency contact
  ↓ (Fails: unreachable)
Result: Log failed escalation, notify user dashboard
```

LLM Fallback:

```
try:
    decision = escalation_engine.decide_escalation(alarm_id)
except APIError:
    # Fallback ladder (UX-01), not another LLM call
    decision = escalation_engine.fallback_ladder(alarm_id)
    log.warning(f"LLM failed, using fallback: {decision}")
```

Why This Design:

- **Reliability:** System works even if external services fail
- **User Trust:** Won't fail silently
- **Debugging:** Know when/why fallbacks happen
- **Graceful Degradation:** Best-effort service

Alternatives Considered:

1. Fail hard (no fallbacks):

- **✗** User misses alarm if service unavailable
- **✗** Bad user experience

Chosen: Comprehensive error handling + fallbacks

User Experience Decisions

UX-01: Fallback escalation ladder (not the primary AI path)

Decision: Primary path (UC-03): `AgentController` + `EscalationEngine.decide_escalation()` let the LLM choose the next channel from enabled methods using `UserProfile` / `EscalationLog`.

Fallback ladder (used when the LLM is down, returns invalid JSON, or picks a disabled method):

1. Alarm sound + verification (already failed / timed out)
2. Phone call
3. SMS
4. Alexa alert
5. Emergency contact (last resort, consent required)

This ladder is a **deterministic safety net**, not the normal AI policy. See also AD-05.

Why This Design:

- AI can personalize order; system still wakes the user if the LLM fails
- Emergency contact stays last even in the fallback path
- Matches UC-03 “AI chooses” without contradicting reliability requirements

Alternatives Considered:

1. **Fixed ladder only (no AI):** rejects the agent requirement
2. **AI only, no fallback:** alarm can be silently dropped on API outage

Chosen: AI decision first; progressive ladder as fallback only

UX-02: Intelligent Check-in (Not Immediate Dismissal)

Decision: After verifying task, send check-in 5-10 min later instead of immediate dismissal.

Why:

- **Prevents Sleeping Again:** Checks user is actually awake (not just fumbled phone)
- **Learning:** More data points about user's wakefulness
- **Better UX:** User can "prove" wakefulness if needed

Flow:

7:00 AM: Alarm triggers
7:02 AM: User completes photo verification
System: "Great! Rest well."
7:10 AM: Check-in notification: "Are you still awake?"
7:10 AM: User confirms "Yes"
Success logged!

Alternative (Rejected):

7:00 AM: Alarm triggers
7:02 AM: User completes photo verification
7:02 AM: Alarm dismissed immediately
(User could go back to bed)

Chosen: Delayed check-in

UX-03: Reminders with Escalation Levels

Decision: Send multiple reminder notifications (primary, escalation, final) rather than single reminder.

Reminder Levels:

9:00 AM: Primary reminder: "Call Mom"
9:05 AM: Escalation: "Hey! Still need to call Mom."
9:20 AM: Final: "Last reminder: Call Mom (High Priority)"

Why:

- **Adaptive:** Can skip escalation reminders if user completes immediately
- **Respects Time:** Don't spam if user completes early
- **Effective:** Multiple nudges increase completion rate
- **Learning:** Track how many reminders needed per task type

Alternatives Considered:

1. Single reminder:

-  User might miss it (forget phone, distracted)
-  No escalation for important tasks

2. Fixed multiple reminders:

-  Spammy (even if user already completed)

Chosen: Adaptive escalation reminders

Summary: Design Principles Applied

Principle	Application
Single Responsibility	Each class has one reason to change

Principle	Application
Open/Closed	New verification tasks, escalation methods via inheritance
Liskov Substitution	Any VerificationTask can replace another
Interface Segregation	LLMClient, ToolManager expose only needed methods
Dependency Inversion	Services depend on abstractions (LLMClient), not concrete APIs
DRY	Shared logic in base classes/services
YAGNI	Only implement what's needed (no unused features)
Composition over Inheritance	Services composed together, not deep hierarchies

Design Evolution & Future Improvements

Possible Enhancements (Not in Stage 1)

1. Machine Learning Model (Once Data Collected)

- After 6 months of data, train ML model on escalation success
- Replace rule-based fallback with ML-based decisions

2. Wearable Integration

- Add support for smartwatch (detect if user actually got out of bed)
- Integrate with fitness trackers for sleep quality data

3. Calendar Integration

- Check Google Calendar before scheduling alarm (don't alarm on days off)
- Prioritize reminders based on calendar events

4. Natural Language Commands

- Voice assistant: "Alexa, ask RiseUp to remind me to..."
- Improve NLP parsing for complex commands

5. Microservices Architecture

- If traffic grows, split into separate services
- Independent scaling for alarms vs. reminders vs. escalation

6. Distributed Scheduling

- If single server can't handle volume, use distributed scheduler
- Ensure alarms trigger reliably at scale

Conclusion

The RiseUp architecture balances:

- **Simplicity:** Easy to understand and implement (Stage 1-2)

- **Extensibility:** Easy to add new features (verification tasks, escalation methods)
- **Maintainability:** Clear separation of concerns, testable components
- **Scalability:** Can grow from single server to multiple services if needed
- **Reliability:** Fallbacks, error handling, ACID database
- **Intelligence:** AI integration without over-relying on LLM

RiseUp - Design Patterns

Chapter 5 — source: docs/stage-1-design/design-patterns.md

Chapter overview. Chapter 5 documents the **five design patterns** RiseUp claims: Strategy, Adapter, Command, Observer and Factory Method. For each it gives the design problem, the participating classes and their roles, why the pattern fits, and what the design would look like without it. It also lists the patterns that are deliberately *not* claimed (Delegation, Repository, Facade on `EscalationEngine`, `State`) and explains why.

[Chapter 8 — Pattern coverage](#) maps every pattern to the features it supports.

Stage 1 requires at least **five** design patterns, each solving a real design problem. RiseUp uses **five**. Patterns claimed elsewhere (Delegation, Repository as a “pattern credit,” `EscalationEngine-as-Facade`, `State`) are **not** counted here — `State` is deferred until `AlarmState` subclasses appear on the class diagram.

Canonical patterns

#	Pattern	Where it appears
1	Strategy	Verification tasks; LLM providers; CLI output formatters
2	Adapter	<code>LLMClient</code> , <code>TwilioClient</code> , <code>AlexaClient</code> , <code>FirebaseClient</code>
3	Command	CLI actions (<code>riseup ...</code>)
4	Observer	Alarm / reminder / check-in status changes → UI & loggers
5	Factory Method	<code>VerificationTaskFactory</code> , <code>ReminderFactory</code>

1. Strategy

Design problem

Verification challenges differ (photo, math, trivia, custom) but the alarm flow must treat them the same. LLM providers also differ (Claude vs OpenAI) while callers only need “send prompt → get text.” CLI output can be a table or JSON.

Participating classes and roles

Class	Role
<code>VerificationTask</code>	Strategy interface (<code>validate()</code> , <code>get_task_prompt()</code>)
<code>ObjectDetectionTask</code> , <code>MathTask</code> , <code>TriviaTask</code> , <code>CustomTask</code>	Concrete verification strategies
<code>LLMProvider</code>	Strategy interface (<code>complete(prompt) -> str</code>)
<code>ClaudeProvider</code> , <code>OpenAIProvider</code>	Concrete LLM strategies
<code>OutputFormatter</code>	Strategy interface for CLI formatting

Class	Role
TableFormatter , JsonFormatter	Concrete format strategies
Alarm / CLIApp / LLMClient	Context that selects and invokes a strategy

Why appropriate

Open/Closed: new task types, providers, or formats are added as new classes without editing callers. Call sites stay polymorphic (`task.validate(response)`).

Without the pattern

Large `if/elif` chains on task type / provider / `--json` flag, duplicated validation logic, and every new variant requiring edits in multiple controllers.

2. Adapter

Design problem

Twilio, Alexa, Firebase, and Anthropic/OpenAI each expose different SDKs and payload shapes. Domain code must not depend on those vendor APIs.

Participating classes and roles

Class	Role
Tool	Target interface used by the domain (<code>execute(payload) -> ToolResult</code>)
TwilioClient	Adapts Twilio REST API to Tool (call / SMS)
AlexaClient	Adapts Alexa notifications to Tool
FirebaseClient	Adapts FCM push to Tool
LLMClient	Adapts LLMProvider responses into domain DTOs via ResponseParser
ToolManager	Client that only talks to Tool / LLMClient

Why appropriate

Isolates vendor churn behind a stable interface; tests mock adapters instead of live APIs.

Without the pattern

EscalationEngine and schedulers would call Twilio/Alexa/Anthropic SDKs directly, coupling business logic to vendor libraries and making unit tests require network stubs everywhere.

3. Command

Design problem

The CLI must expose many actions (`alarm add` , `remind` , `stats` , ...) with uniform parsing, exit codes, and testability, without growing a giant `if` in `main` .

Participating classes and roles

Class	Role
<code>Command</code>	Command interface (<code>execute(args) -> Result</code>)
<code>AddAlarmCommand</code> , <code>RemindCommand</code> , <code>StatsCommand</code> , ...	Concrete commands
<code>CommandRegistry</code>	Maps <code>argv verb</code> → <code>Command</code>
<code>CLIApp</code>	Invoker (parse <code>argv</code> , look up command, run, set exit code)
<code>APIClient</code>	Receiver helper (HTTP to FastAPI)

Why appropriate

Each CLI action is an object that can be unit-tested alone; new verbs register without changing `CLIApp` .

Without the pattern

One monolithic CLI switch statement, hard-to-test side effects, and tangled error handling.

4. Observer

Design problem

When an alarm rings, a reminder is missed, or a check-in times out, several parts must react (dashboard banner, history log, escalation trigger) without the domain objects knowing about UI.

Participating classes and roles

Class	Role
<code>Alarm</code> / <code>Reminder</code> / <code>CheckIn</code>	Subjects that change status and notify
<code>ReminderObserver</code> (interface)	Observer contract (<code>on_status_changed(event)</code>)
<code>DashboardNotifier</code>	Updates GUI / push state
<code>ReminderLogger</code> / <code>EscalationLog</code> writer	Persists history
<code>EscalationEngine</code> (as listener on timeout)	Starts agent escalation when check-in fails

Why appropriate

Subjects emit events; observers subscribe. UI and logging can change without editing `Alarm` .

Without the pattern

`Alarm.stop()` would hard-code UI updates, logging, and escalation calls (tight coupling, harder to test, easy to miss a side effect when adding a new consumer).

5. Factory Method

Design problem

Creating the right `VerificationTask` or `Reminder` depends on user prefs, AI-parsed fields, or fallbacks. Callers should not `new` concrete classes with scattered construction rules.

Participating classes and roles

Class	Role
<code>VerificationTaskFactory</code>	<code>create(task_type, prefs) -> VerificationTask</code>
<code>ReminderFactory</code>	<code>`create(parsed_fields</code>
<code>Alarm</code> / <code>NLCommandParser</code> flow	Clients that ask factories for products

Why appropriate

Centralizes construction, defaults, and fallbacks (e.g. camera unavailable → `MathTask`).

Without the pattern

Construction logic duplicated in GUI controllers, CLI commands, and schedulers; inconsistent defaults and harder testing of creation rules.

Patterns explicitly not claimed

Name	Why not claimed
Delegation	Useful technique inside <code>ToolManager</code> , but not used as a Stage 1 pattern credit
Repository	Repositories exist for persistence; not counted as one of the five course patterns
Facade on <code>EscalationEngine</code>	<code>EscalationEngine</code> is an orchestrator used by <code>AgentController</code> , not a thin Facade over a subsystem
State	Alarm lifecycle is real, but the class diagram does not yet show <code>AlarmState</code> subclasses; do not claim State until those classes are on the diagram

`APIClient` may still *act* as a small Facade for HTTP/auth; it is not needed for the five-pattern minimum.

RiseUp - Feature-to-Design Traceability

Chapter 6 — source: docs/stage-1-design/feature-to-design-mapping.md

Chapter overview. Chapter 6 links the requirements to the design. It contains the sequence-diagram ID map (SD-01 – SD-07), a traceability table that maps every feature to its use cases, classes, key methods, sequence diagram and patterns, and *Task 4*: a short explanation of how each of the 14 features is realised by collaborating classes. It ends with the agent loop pseudo-code shared by F04, F05, F07 and F13.

[Chapter 8](#) extends this table with CLI commands, decision references and fallbacks.

Canonical features: **F01–F14** in [feature-specification.md](#) . Patterns: [design-patterns.md](#) . Sequence IDs: see table below.

Sequence diagram ID map

ID	File	Covers
SD-01	SequenceDiagramAlarm.png	F01–F03 alarm, verify, check-in
SD-02	SequenceDiagramAi.png	F04–F05 agent escalation loop
SD-03	SequenceDiagramTaskReminder.png	F08–F10 reminders + prioritization
SD-04	SequenceDiagramLearningEngine.png	F11 learning
SD-05	SequenceDiagramCliRemind.png	F07 CLI/NL remind (GUI same backend path)
SD-06	SequenceDiagramRecommendations.png	F13 recommendations
SD-07	SequenceDiagramHistoryExplain.png	F14 history / explain
—	Use F06 with UC-08 prefs UI (deterministic; no dedicated SD beyond prefs save)	F06
—	F12 uses analytics query path (shown conceptually with F11/F13 data)	F12 → treat SD-04 + dashboard

F06 and F12 are deterministic CRUD/query flows; their collaboration is described in Task 4 below and uses the same controller → repository pattern as F01/F08. Graders can treat SD-01/SD-04 as the structural template.

Feature-to-Design Traceability Table

Feature	Description	Type	Related UC	Classes	Key Methods	Sequence	Pattern(s)
F01	Alarm Management	Deterministic	UC-01	AlarmController, Alarm, AlarmRepository, AlarmScheduler, UserPreferences	create_alarm(), validate(), save(), schedule_alarm()	SD-01	—
F02	Verification Challenge	Deterministic	UC-02	VerificationTaskFactory, VerificationTask, ObjectDetectionTask, MathTask, TriviaTask, CustomTask, Alarm	create(), validate(), get_task_prompt(), on_verify()	SD-01	Strategy, Factory Method
F03	Post-Wake Check-in	Deterministic	UC-02	CheckIn, ToolManager, FirebaseClient, Alarm	schedule(), send(), record_response(), time_out_handler()	SD-01	Observer, Adapter
F04	AI Escalation Decision	AI/Hybrid	UC-03	AgentController, EscalationEngine, PromptBuilder, LLMClient, LLMProvider, ResponseParser, UserProfile, EscalationLog	run(), decide_escalation(), build_escalation_prompt(), send_request(), parse_decision()	SD-02	Strategy (provider), Adapter
F05	Multi-Channel Escalation Execution	Deterministic	UC-03	ToolManager, Tool, TwilioClient, AlexaClient, EscalationLog, UserProfile	execute_escalation(), make_call(), send_sms(), send_alert(), record_escalation()	SD-02	Adapter, Observer

Feature	Description	Type	Related UC	Classes	Key Methods	Sequence	Pattern(s)
F06	Emergency Contact & Prefs	Deterministic	UC-08	PreferencesController, UserPreferences, EmergencyContact, ToolManager	update(), validate(), request_consent(), send test	SD-01 style	—
F07	NL Reminder Creation	Hybrid	UC-04	CLIApp / ReminderCommand or GUI, NLCommandParser, PromptBuilder, LLMClient, ResponseParser, ReminderFactory, ReminderScheduler	parse(), send_request(), parse_reminder(), create(), schedule_reminder()	SD-05	Command (CLI), Factory, Strategy
F08	Reminder Management	Deterministic	UC-04, UC-06	ReminderController, Reminder, ReminderRepository, ReminderFactory, ReminderScheduler	validate(), save(), schedule_reminder(), mark_complete()	SD-03	Factory Method
F09	AI Reminder Prioritization	AI/Hybrid	UC-05	TaskPrioritizer, LLMClient, ResponseParser, UserProfile, ReminderScheduler	prioritize_reminders(), send_request(), parse_ranking()	SD-03	Strategy (provider)
F10	Reminder Escalation Levels	Deterministic	UC-05	ReminderScheduler, Reminder, ToolManager, ReminderObserver	send_primary_reminder(), send_escalation_reminder(), send_final_reminder(), on_status_changed()	SD-03	Observer

Feature	Description	Type	Related UC	Classes	Key Methods	Sequence	Pattern(s)
F11	Behavioral Learning	Deterministic	UC-03, UC-05, UC-07	LearningEngine, AnalyticsEngine, UserProfile, EscalationLog	analyze_behavior(), compute_metrics(), update_learning_data()	SD-04	—
F12	Analytics Dashboard	Deterministic	UC-07	AnalyticsController, AnalyticsEngine, UserProfile, OutputFormatter	get_stats(), query(), format()	SD-04 / CLI	Strategy (formatter)
F13	AI Recommendations	AI	UC-07	RecommendationService, AgentController, PromptBuilder, LLMClient, ResponseParser, UserPreferences	generate(), run(), parse_recommendations(), update()	SD-06	Strategy, Adapter
F14	History & Explain	Hybrid	UC-09	HistoryController, EscalationLog, ReminderLog	list(), explain()	SD-07	—

Dropped from Stage 1 credit: “Web Application Interface,” bare “Alarm Scheduling,” Delegation, Repository-as-pattern, EscalationEngine-as-Facade.

Task 4 — How each feature is realized

F01 — Alarm Management

UC: UC-01 · **SD:** SD-01

`AlarmController` accepts GUI/CLI input → `Alarm.validate()` → `AlarmRepository.save()` → `AlarmScheduler.schedule_alarm()` places the job; alarm is stored as active/scheduled.

F02 — Verification Challenge

UC: UC-02 · **SD:** SD-01

On trigger, the alarm rings. `VerificationTaskFactory.create()` returns a Strategy task. GUI (or CLI for non-photo) collects a response; `validate()` succeeds → alarm stops and check-in is scheduled; failures retry; camera failure → factory emits `MathTask`.

F03 — Post-Wake Check-in

UC: UC-02 · **SD:** SD-01

After verify, `CheckIn.schedule()` ; `ToolManager` + `FirebaseClient` notify. Response → `record_response()` ; timeout notifies observers and starts F04.

F04 — AI Escalation Decision

UC: UC-03 · **SD:** SD-02

`AgentController.run("escalate", ctx)` observes `UserProfile` + `EscalationLog` , `EscalationEngine.decide_escalation()` builds a prompt, `LLMClient` / `LLMProvider` answers, `ResponseParser` validates method ∈ enabled list. On LLM failure → UX-01 fallback ladder.

F05 — Escalation Execution

UC: UC-03 · **SD:** SD-02

`ToolManager.execute_escalation(method)` uses `Tool` adapters. Outcome logged; `UserProfile.update_learning_data()` . If tool fails, agent **re-decides** or walks fallback.

F06 — Prefs & Emergency Contact

UC: UC-08 · **SD:** prefs save (same shape as F01)

`PreferencesController` updates `UserPreferences` ; `EmergencyContact.request_consent()` must succeed before that contact is eligible for F05.

F07 — NL Reminder Creation

UC: UC-04 · **SD:** SD-05

CLI `RemindCommand` or GUI quick-add → `NLCommandParser` → LLM JSON → user confirms → `ReminderFactory.create()` → schedule. Ambiguous/failed parse → clarifying question or manual form.

F08 — Reminder Management

UC: UC-04, UC-06 · **SD:** SD-03

CRUD via `ReminderController` / repository / scheduler; complete/snooze/delete update state and observers.

F09 — AI Prioritization

UC: UC-05 · **SD:** SD-03

When multiple due, `TaskPrioritizer.prioritize_reminders()` uses LLM ranking validated to be a permutation of IDs; else deterministic sort by priority → category → due time.

F10 — Reminder Escalation Levels

UC: UC-05 · **SD:** SD-03

`ReminderScheduler` sends primary / +5m / +15m; `ReminderObserver` updates UI/logs; complete cancels the chain.

F11 — Behavioral Learning

UC: related to UC-03/05/07 · **SD:** SD-04

Nightly `LearningEngine.analyze_behavior()` → `AnalyticsEngine` → `UserProfile` metrics used as agent memory on next F04/F09/F13.

F12 — Analytics Dashboard

UC: UC-07 · **SD:** SD-04 data path

`AnalyticsController.get_stats(range)` reads aggregates; GUI charts or CLI `OutputFormatter (--json)`.

F13 — AI Recommendations

UC: UC-07 · **SD:** SD-06

`RecommendationService.generate()` via agent/LLM; each suggestion validated against settings schema; Accept applies `UserPreferences.update()`.

F14 — History & Explain

UC: UC-09 · **SD:** SD-07

`HistoryController.list/explain` reads stored reasoning/confidence from F04/F09 decisions (including “fallback used” flag).

Agent loop (applies to F04, F05, F07 confirm side-effects, F13)

```
observe(memory: UserProfile, EscalationLog, last ToolResult)
  → decide(LLM structured action)
  → act(ToolManager)
  → observe(outcome)
  → if failed and retries left: decide again
  → else: deterministic fallback / stop
```

`EscalationEngine` **orchestrates** escalation decisions; it is not claimed as a Facade pattern.

RiseUp - UML Diagrams

Chapter 7 — source: docs/stage-1-design/uml-diagrams/

Chapter overview. This chapter contains every UML diagram produced in Stage 1. Each diagram is embedded at full width and followed by a walkthrough: the participants, the message flow step by step, every alternative (`alt`) path, and the features, use cases, decisions and patterns it realises. Sequence diagram IDs (SD-01 – SD-07) match [Chapter 6](#).

Diagram Index

#	ID	Diagram	File	Covers
1	—	Architecture Diagram	ArchitectureDiagram.png	All features (F01–F14)
2	—	Class Diagram	ClassDiagram.png	All features (F01–F14)
3	—	Use Case Diagram	UseCaseDiagram.png	UC-01 – UC-09
4	SD-01	Sequence Diagram — Alarm / Verify / Check-in	SequenceDiagramAlarm.png	F01, F02, F03 (template for F06)
5	SD-02	Sequence Diagram — Agent Escalation Loop	SequenceDiagramAi.png	F04, F05
6	SD-03	Sequence Diagram — Reminders & Prioritization	SequenceDiagramTaskReminder.png	F08, F09, F10
7	SD-04	Sequence Diagram — Behavioral Learning	SequenceDiagramLearningEngine.png	F11 (and F12 analytics query path)
8	SD-05	Sequence Diagram — CLI / Natural-Language Remind	SequenceDiagramCliRemind.png	F07
9	SD-06	Sequence Diagram — AI Recommendations	SequenceDiagramRecommendations.png	F13
10	SD-07	Sequence Diagram — History & Decision Explanation	SequenceDiagramHistoryExplanation.png	F14

7.1 Architecture Diagram

- **File:** `uml-diagrams/ArchitectureDiagram.png`
- **Purpose:** Overall system architecture: React GUI and `riseup` CLI clients, the FastAPI backend (controllers/services, `AgentController`, `EscalationEngine`, `TaskPrioritizer`, `LearningEngine`), the LLM layer (Claude / OpenAI behind `LLMClient`), external integrations (Twilio, Alexa, Camera, Firebase) and the data layer (PostgreSQL / Redis).
- **Covers:** All features (F01–F14)
- **Related:** Project Scope §5, Design Decisions (Architectural Decisions)

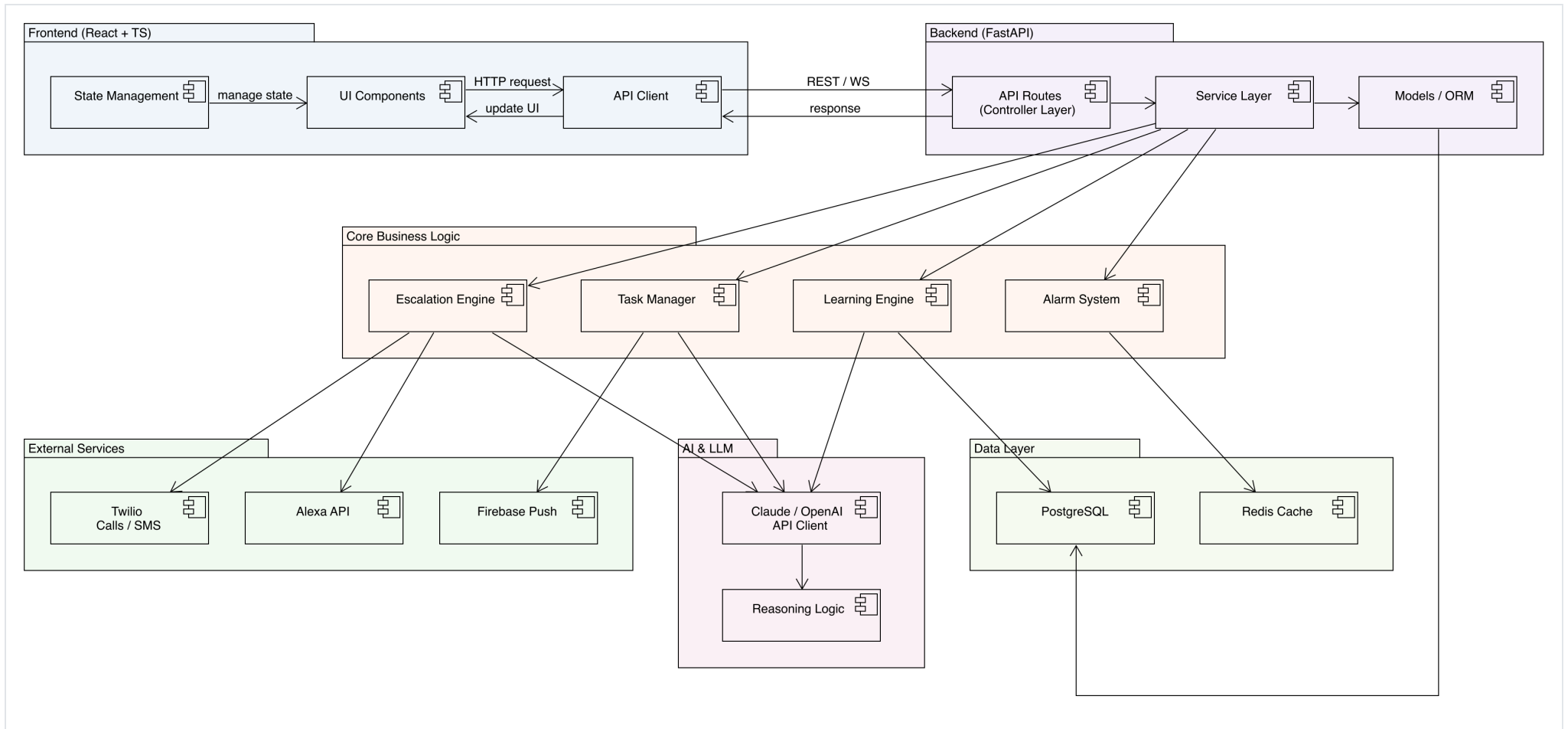


Figure 1: Architecture Diagram

Walkthrough

What the diagram shows. The deployment-level structure of RiseUp, grouped into six colour-coded areas.

Area	Elements	Responsibility
Frontend (React + TS)	State Management, UI Components, API Client	Render the dashboard; manage client state; send HTTP requests and receive REST/WebSocket responses
Backend (FastAPI)	API Routes / Controller Layer → Service Layer → Models/ORM	Accept requests, run business services, map objects to tables
Core Business Logic	Escalation Engine, Task Manager, Learning Engine, Alarm System	Domain behaviour: escalation, reminder handling and prioritisation, behavioural learning, alarm triggering
External Services	Twilio (calls/SMS), Alexa API, Firebase Push	Channels used to reach the user
AI & LLM	Claude / OpenAI API client → Reasoning Logic	Structured LLM calls used for decisions
Data Layer	PostgreSQL, Redis Cache	Durable storage and fast cache / real-time messaging

Main flows.

1. The **API Client** in the browser sends requests over **REST/WebSocket** to the **API Routes**. Responses return to the API Client, which updates the UI components.
2. The **Service Layer** delegates to the four core modules and to **Models/ORM**.
3. The **Escalation Engine** uses Twilio and Alexa to reach the user and calls the LLM client to decide which channel to use.
4. The **Task Manager** uses Firebase Push for reminder notifications and the LLM client for prioritisation.
5. The **Learning Engine** uses the LLM client and PostgreSQL. The **Alarm System** persists to PostgreSQL and uses Redis.

Design rationale. This is the layered client-server architecture of **AD-01**: UI, business logic and data are separated, so each layer can be tested and scaled on its own. The CLI (not drawn here) reaches the same API Routes, as shown in the CLI design in [Chapter 2](#).

7.2 Class Diagram

- **File:** `uml-diagrams/ClassDiagram.png`
- **Purpose:** Static structure of the system: domain entities, controllers, services, the CLI command classes, the agent (`AgentController`), LLM providers (`LLMClient` / `LLMProvider`), and the tool adapters (`ToolManager` , `TwilioClient` , `AlexaClient` , `FirebaseClient`).
- **Covers:** All features (F01–F14)
- **Related:** Design Patterns, Feature-to-Design Traceability

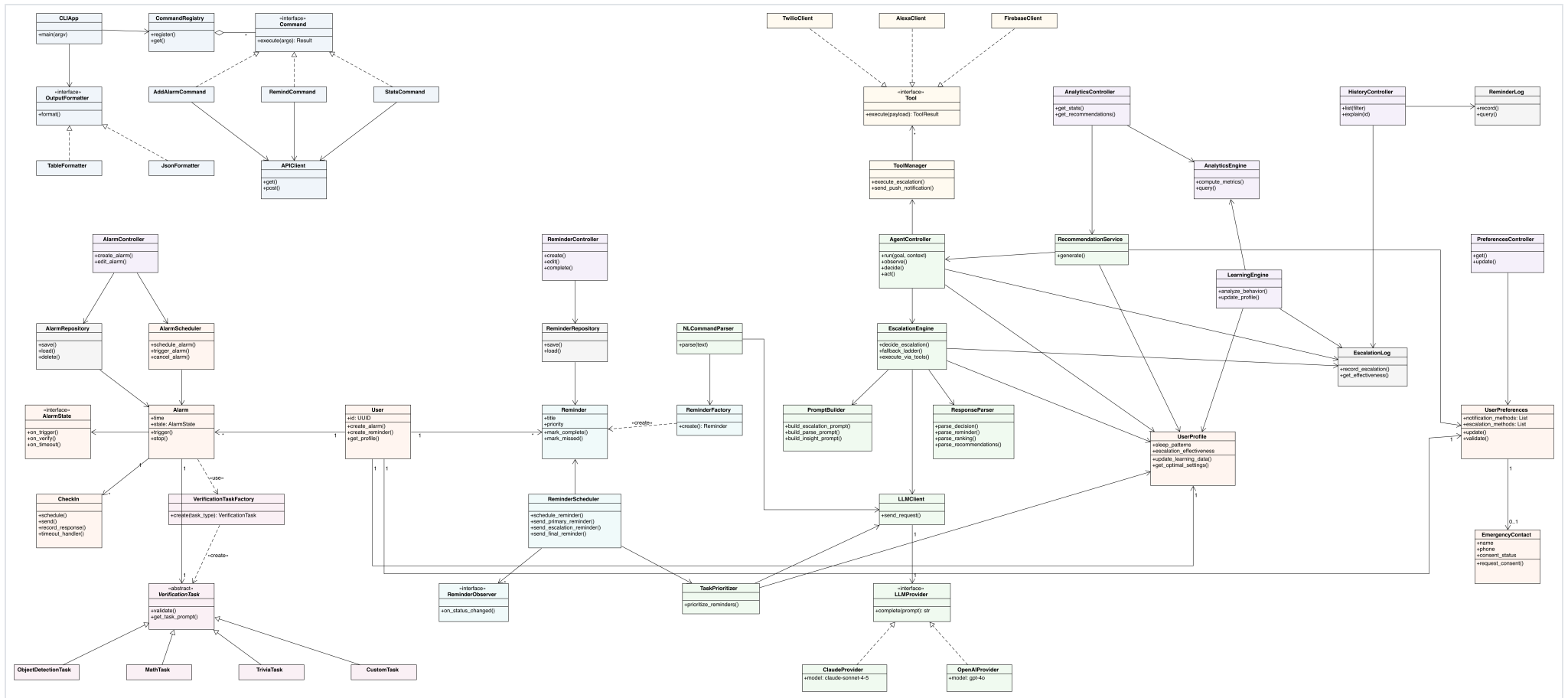


Figure 2: Class Diagram

Walkthrough

What the diagram shows. The static structure of the whole system, including the classes added by the feature specification. The classes are grouped below by layer. Every class named in the traceability table appears on the diagram.

Group	Classes	Notes
CLI	CLIApp, CommandRegistry, Command «interface», AddAlarmCommand, RemindCommand, StatsCommand, APIClient, OutputFormatter «interface», TableFormatter, JsonFormatter	Command pattern (CLIApp invoker, concrete commands) and Strategy pattern (formatters)
Controllers	AlarmController, ReminderController, PreferencesController, AnalyticsController, HistoryController	Thin entry points used by REST routes
Agent & AI	AgentController, EscalationEngine, TaskPrioritizer, RecommendationService, NLCommandParser, PromptBuilder, ResponseParser, LLMClient, LLMPProvider «interface», ClaudeProvider, OpenAIProvider	AgentController runs observe/decide/act; LLMClient + providers = Adapter + Strategy
Tools	ToolManager, Tool «interface», TwilioClient, AlexaClient, FirebaseClient	Adapter pattern over vendor SDKs
Domain	User, UserProfile, UserPreferences, EmergencyContact, Alarm, CheckIn, Reminder	Core entities; UserProfile is the agent's long-term memory
Verification	VerificationTask «abstract», ObjectDetectionTask, MathTask, TriviaTask, CustomTask, VerificationTaskFactory	Strategy + Factory Method
Reminders	ReminderFactory, ReminderScheduler, ReminderObserver «interface»	Factory Method + Observer
Scheduling	AlarmScheduler, ReminderScheduler	APScheduler-backed trigger jobs
Learning & analytics	LearningEngine, AnalyticsEngine	Nightly aggregation of logs into UserProfile
Persistence & logs	AlarmRepository, ReminderRepository, EscalationLog, ReminderLog	Storage and history for learning and F14

Key relationships.

- AgentController uses EscalationEngine, ToolManager, UserProfile and EscalationLog.
- EscalationEngine, TaskPrioritizer, RecommendationService and NLCommandParser depend on PromptBuilder, LLMClient and ResponseParser.
- LLMClient holds one LLMPProvider, either ClaudeProvider or OpenAIProvider.
- ToolManager holds Tool implementations: TwilioClient, AlexaClient and FirebaseClient.
- Alarm uses VerificationTaskFactory, which creates one of the four concrete VerificationTask classes.
- Reminder subjects notify ReminderObserver implementations.
- User owns Alarm, Reminder, UserProfile and UserPreferences. UserPreferences references EmergencyContact.

The full list of classes and their responsibilities is in the [class catalogue](#).

7.3 Use Case Diagram

- **File:** `uml-diagrams/UseCaseDiagram.png`
- **Purpose:** Actors and use cases UC-01 – UC-09. Actors: User, System/Scheduler, LLM Service (Claude / OpenAI), Twilio, Alexa, Camera / Vision and Firebase (push).
- **Covers:** UC-01 – UC-09
- **Related:** Use Cases

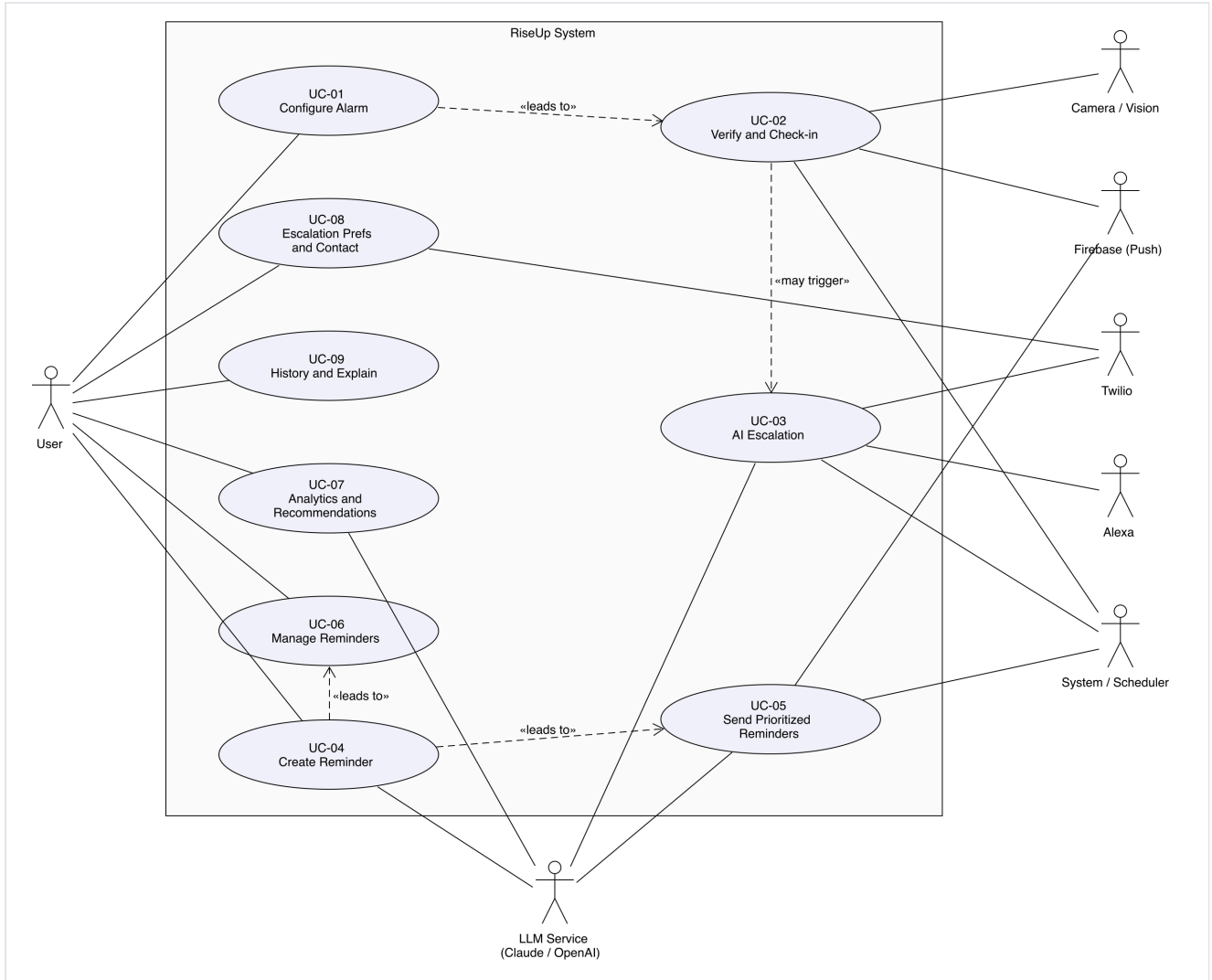


Figure 3: Use Case Diagram

Walkthrough

What the diagram shows. The nine use cases and the actors who take part in each.

Actor	Connected use cases
User	UC-01, UC-04, UC-06, UC-07, UC-08, UC-09
System / Scheduler	UC-02, UC-03, UC-05
LLM Service	UC-03, UC-04, UC-05, UC-07
Twilio	UC-03, UC-08

Actor	Connected use cases
Alexa	UC-03
Camera	UC-02
Firebase	UC-02, UC-05

Dependencies (dotted arrows).

- UC-01 *Configure Alarm* **leads to** UC-02 *Verify and Check-in*.
- UC-02 **may trigger** UC-03 *AI Escalation*, after a verification timeout or a missed check-in.
- UC-04 *Create Reminder* **leads to** UC-05 *Send Prioritized Reminders* and UC-06 *Manage Reminders*.

There is deliberately **no** link from UC-03 to UC-05. Escalation and reminder prioritisation are independent flows that both happen to use the LLM.

7.4 SD-01 — Sequence Diagram — Alarm / Verify / Check-in

- **File:** `uml-diagrams/SequenceDiagramAlarm.png`
- **Purpose:** Alarm creation and scheduling, alarm trigger, verification challenge (created by `VerificationTaskFactory`) and the post-wake check-in.
- **Covers:** F01, F02, F03 (template for F06)
- **Related:** UC-01, UC-02

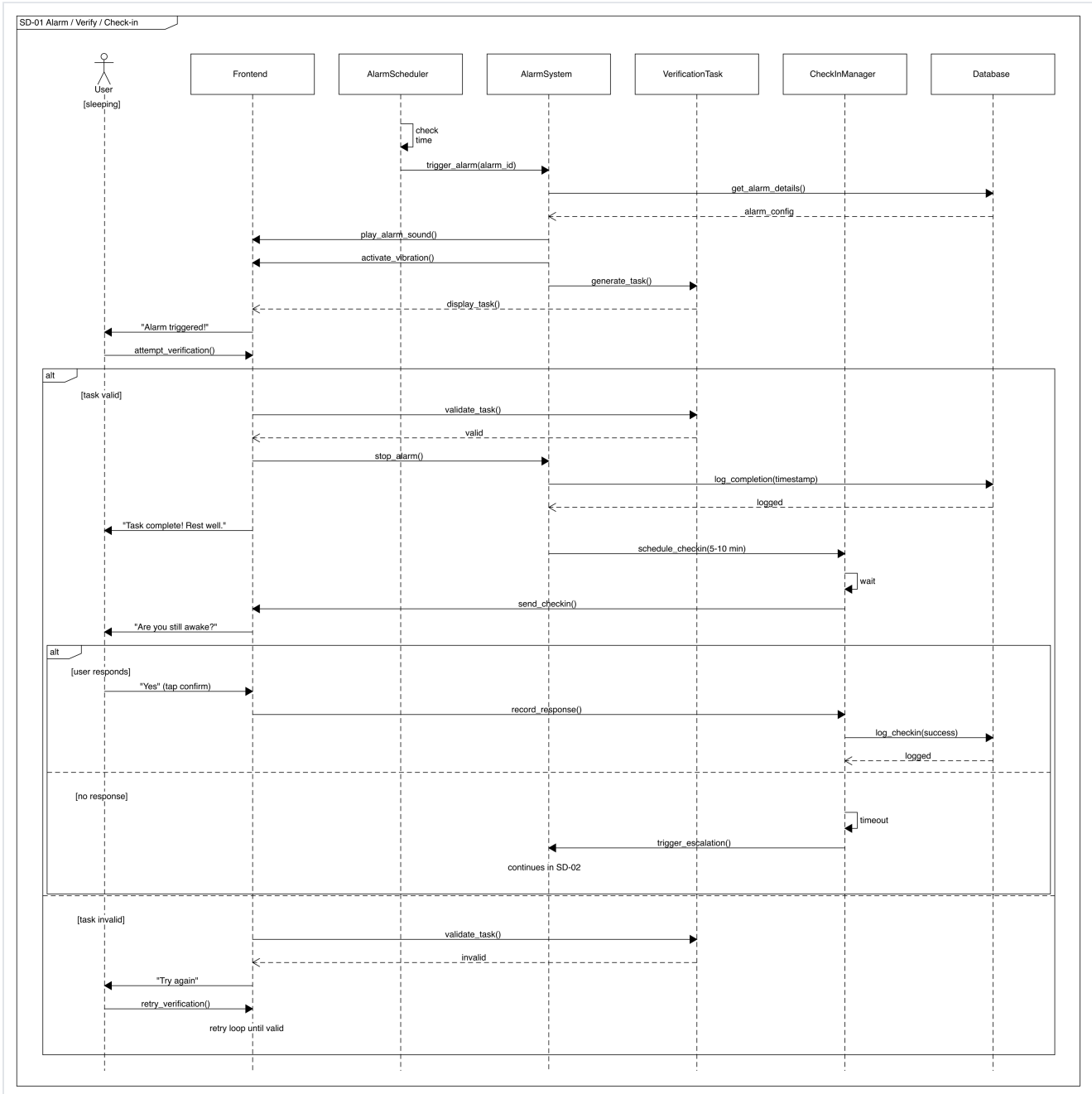


Figure 4: SD-01 — Sequence Diagram — Alarm / Verify / Check-in

Walkthrough

Participants: User, Frontend, AlarmScheduler, AlarmSystem, Verification Task, Check-in Manager, Database.

Main flow.

1. While the user sleeps, `AlarmScheduler` checks whether the current time equals the alarm time and calls `trigger_alarm(alarm_id)`.
2. `AlarmSystem` loads `get_alarm_details()` from the database. It then tells the Frontend to `play_alarm_sound()` and `activate_vibration()`.
3. `AlarmSystem` calls `generate_task()` on Verification Task, and the task is shown in the Frontend with `display_task()`.
4. The user sees "*Alarm triggered!*" and calls `attempt_verification()`.

alt [Task Valid]. The Frontend calls `validate_task()` and gets . It then calls `stop_alarm()`, and `AlarmSystem` runs `log_completion(timestamp)` in the database. The user sees "*Task Complete! Rest well.*". `AlarmSystem` calls `schedule_checkin(5-10 min)`. The Check-in Manager waits, then calls `send_checkin()`, and the user is asked "*Are you still awake?*".

- **[User Responds]:** the user taps confirm. The Frontend calls `record_response()`, followed by `log_checkin(success)`.
- **[User Doesn't Respond]:** after `timeout(5 min)` the Check-in Manager calls `trigger_escalation()` on `AlarmSystem`, which continues in **SD-02**.

alt [Task Invalid]. The task is rejected and the user sees "*Try again*" and calls `retry_verification()`. This retry loop continues until the task is valid.

Requirements realised: F01 (scheduled alarm), F02 (verification challenge, including retries) and F03 (check-in and its timeout) for UC-01 and UC-02. **Patterns:** Strategy and Factory Method for verification tasks; Observer for the check-in timeout.

7.5 SD-02 — Sequence Diagram — Agent Escalation Loop

- **File:** `uml-diagrams/SequenceDiagramAi.png`
- **Purpose:** The `AgentController` observe → decide → act loop: building the prompt, querying the LLM, parsing the decision, executing the escalation through `ToolManager` (call / SMS / Alexa / emergency contact), logging, and deterministic fallback.
- **Covers:** F04, F05
- **Related:** UC-03

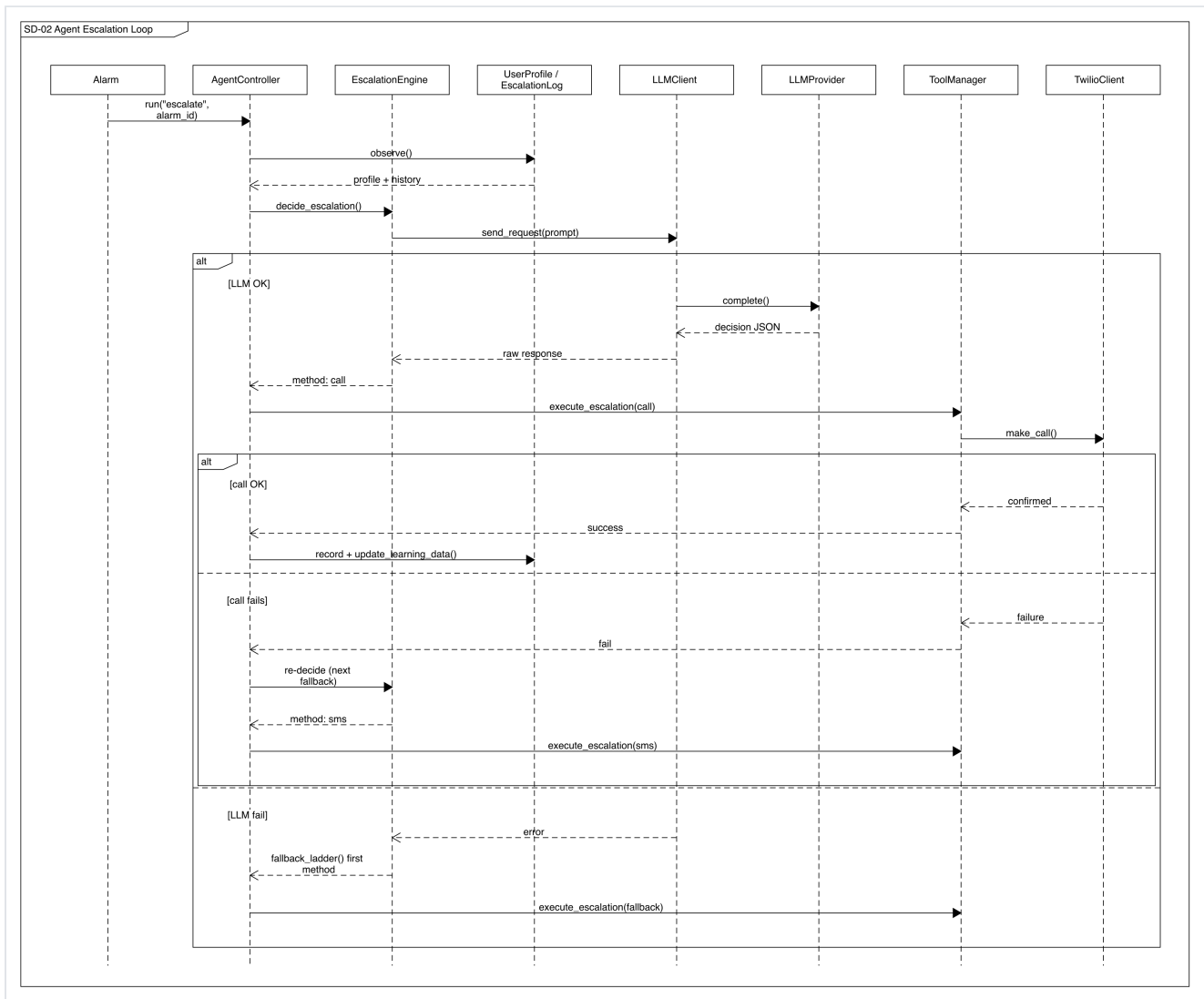


Figure 5: SD-02 — Sequence Diagram — Agent Escalation Loop

Walkthrough

Participants: Alarm, AgentController, EscalationEngine, UserProfile_EscalationLog, LLMClient, LLMProvider, ToolManager, TwilioClient.

Main flow.

1. Alarm calls `run("escalate", alarm_id)` on AgentController.

2. **Observe:** `AgentController` calls `observe()` and receives profile and history from `UserProfile / EscalationLog`.
3. **Decide:** `AgentController` calls `decide_escalation()` on `EscalationEngine`, which calls `LLMClient.send_request(prompt)`.

alt [LLM OK]. `LLMClient` calls `LLMProvider.complete()` and gets the decision JSON. The raw response goes back to `EscalationEngine`, which returns `method=call`. **Act:** `AgentController` calls `ToolManager.execute_escalation(call)`, which calls `TwilioClient.make_call()`.

- **[call OK]:** the call is confirmed and success is returned. **Observe outcome:** `AgentController` calls `record + update_learning_data()`.
- **[call fails]:** the call fails and the failure is returned. `AgentController` **re-decides** with the next fallback, `EscalationEngine` returns `method=sms`, and `execute_escalation(sms)` runs.

alt [LLM fail]. `LLMClient` returns an error. `EscalationEngine` uses `fallback_ladder()` and returns its first method, and `AgentController` calls `execute_escalation(fallback)`.

Requirements realised: F04 (AI escalation decision) and F05 (multi-channel execution) for UC-03, implementing **AD-05** (agent loop), **TR-02** and **UX-01** (fallback ladder). **Patterns:** Adapter (`LLMClient`, `TwilioClient`) and Strategy (`LLMProvider`).

7.6 SD-03 — Sequence Diagram — Reminders & Prioritization

- **File:** `uml-diagrams/SequenceDiagramTaskReminder.png`
- **Purpose:** Reminder creation and storage, AI prioritization via `TaskPrioritizer`, and the reminder escalation levels (primary → escalation → final) with observers notified on status change.
- **Covers:** F08, F09, F10
- **Related:** UC-04, UC-05, UC-06

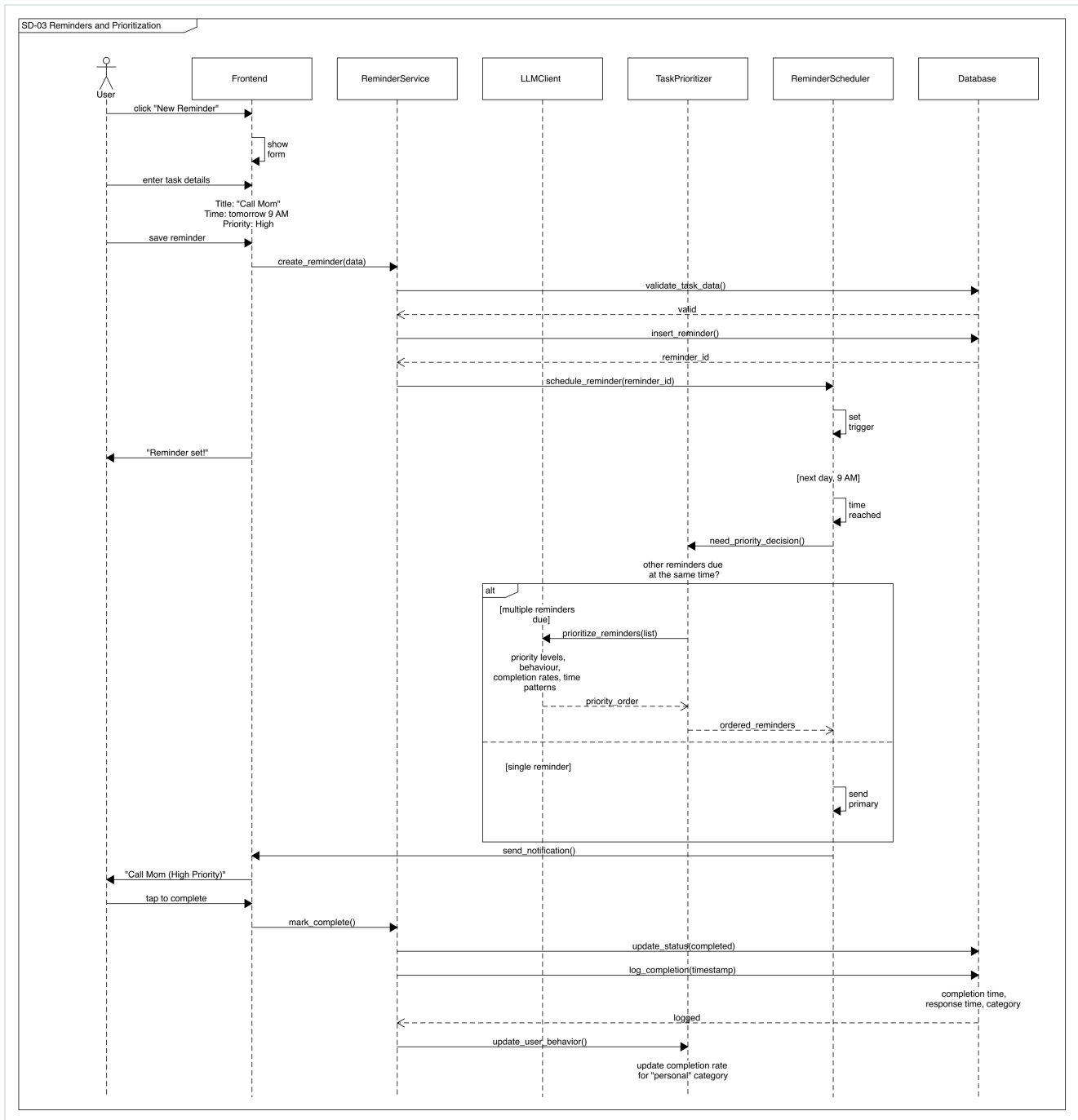


Figure 6: SD-03 — Sequence Diagram — Reminders & Prioritization

Walkthrough

Participants: User, Frontend, ReminderService, LLMClient, TaskPrioritizer, ReminderScheduler, Database.

Creation.

1. The user clicks *New Reminder* and the Frontend shows the form. The user enters the details, for example *Title: "Call Mom", Time: Tomorrow 9 AM, Priority: High*, and saves.
2. The Frontend calls `create_reminder(data)` on `ReminderService`. `ReminderService` calls `validate_task_data()` (✓ Valid), then `insert_reminder()`, which returns the `reminder_id`, then `schedule_reminder(reminder_id)`.
3. `ReminderScheduler` calls `set_trigger_time(tomorrow 9 AM)`, and the user sees "*Reminder set!*".

Delivery. 4. At 9 AM the next day, `time_reached()` fires, and `ReminderScheduler` asks `TaskPrioritizer` `need_priority_decision()`. This checks whether other reminders are due at the same time.

- **alt [Multiple Reminders Due]:** `TaskPrioritizer` calls `prioritize_reminders(list)` on `LLMClient`. The LLM considers priority levels, user behaviour, completion rates and time patterns, and returns `priority_order`. `ReminderScheduler` then receives the `ordered_reminders`.
 - **[Single Reminder]:** the AI step is skipped and `send_primary_reminder()` runs.
5. `ReminderScheduler` calls `send_notification()`, and the user sees "*Call Mom (High Priority)*".

Completion and learning. 6. The user taps to complete. `ReminderService` calls `mark_complete()`, then `update_status(completed)` and `log_completion(timestamp)`. The completion record holds the completion time, response time and category. 7. `ReminderService` calls `update_user_behavior()` on `TaskPrioritizer`, which updates the completion rate for the "personal" category.

Requirements realised: F08 (reminder management), F09 (AI prioritisation, skipped when only one reminder is due) and F10 (the primary reminder; the +5 min and +15 min levels are specified in Chapter 2) for UC-04 and UC-05.

7.7 SD-04 — Sequence Diagram — Behavioral Learning

- **File:** `uml-diagrams/SequenceDiagramLearningEngine.png`
- **Purpose:** `LearningEngine` analyses escalation and completion history, computes metrics with `AnalyticsEngine` and updates the `UserProfile` used in later decisions.
- **Covers:** F11 (and F12 analytics query path)
- **Related:** UC-03, UC-05, UC-07

SD-04 Behavioral Learning

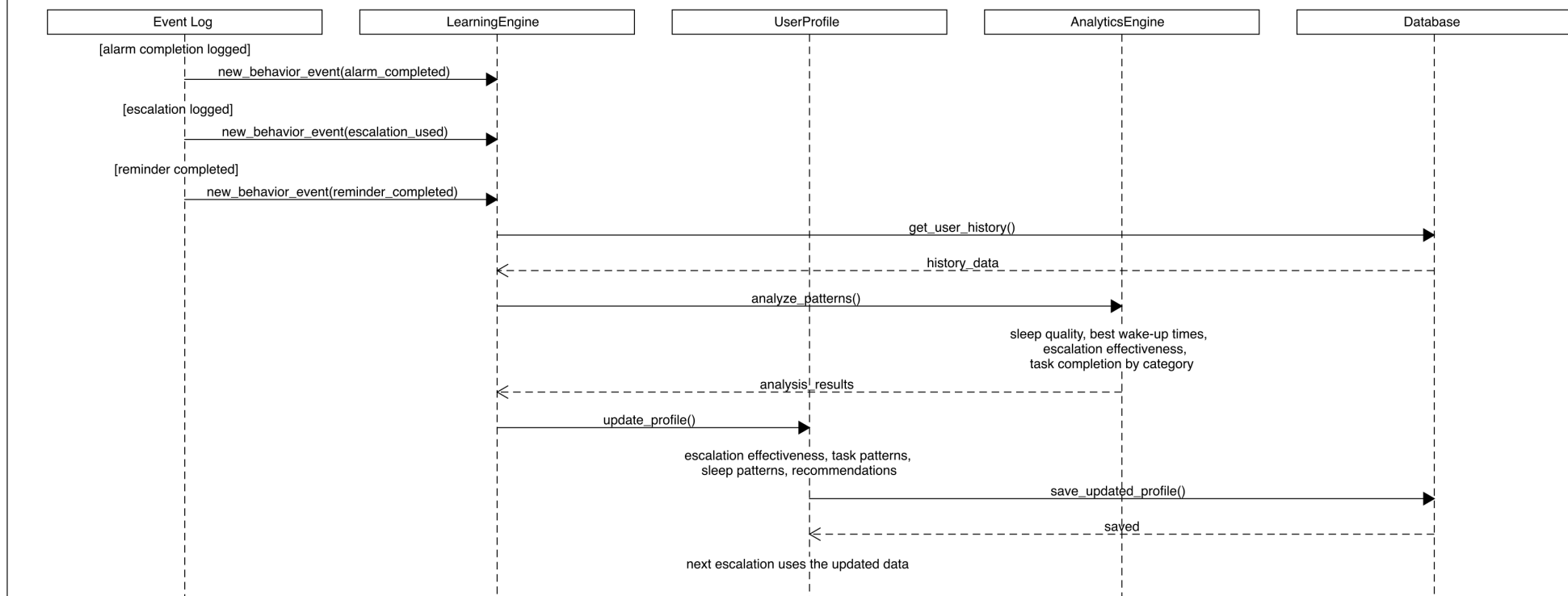


Figure 7: SD-04 — Sequence Diagram — Behavioral Learning

Walkthrough

Participants: Event Log, LearningEngine, UserProfile, AnalyticsEngine, Database.

Main flow.

1. During the day the Event Log sends behaviour events to LearningEngine :
`new_behavior_event(alarm_completed)` , `new_behavior_event(escalation_used)` and `new_behavior_event(reminder_completed)` .
2. LearningEngine calls `get_user_history()` on the database and receives `history_data` .
3. LearningEngine calls `analyze_patterns()` on AnalyticsEngine , which calculates sleep quality, best wake-up times, escalation effectiveness and task completion by category, and returns `analysis_results` .
4. LearningEngine calls `update_profile()` on UserProfile . This updates escalation effectiveness, task patterns, sleep patterns and recommendation inputs.
5. UserProfile calls `save_updated_profile()` and the database returns *saved*.
6. The next escalation uses the updated data.

Requirements realised: F11 (behavioural learning and profile update), and the data path for F12 (analytics) and F13 (recommendations). It implements **DP-03** (pre-aggregated metrics updated nightly). The design is pure statistics, so it can be fully unit-tested.

7.8 SD-05 — Sequence Diagram — CLI / Natural-Language Remind

- **File:** `uml-diagrams/SequenceDiagramCliRemind.png`
- **Purpose:** `riseup remind "..."` flow: CLI command → `NLCommandParser` → LLM → `ResponseParser` → `ReminderFactory` → `ReminderScheduler`. The GUI uses the same backend path.
- **Covers:** F07
- **Related:** UC-04

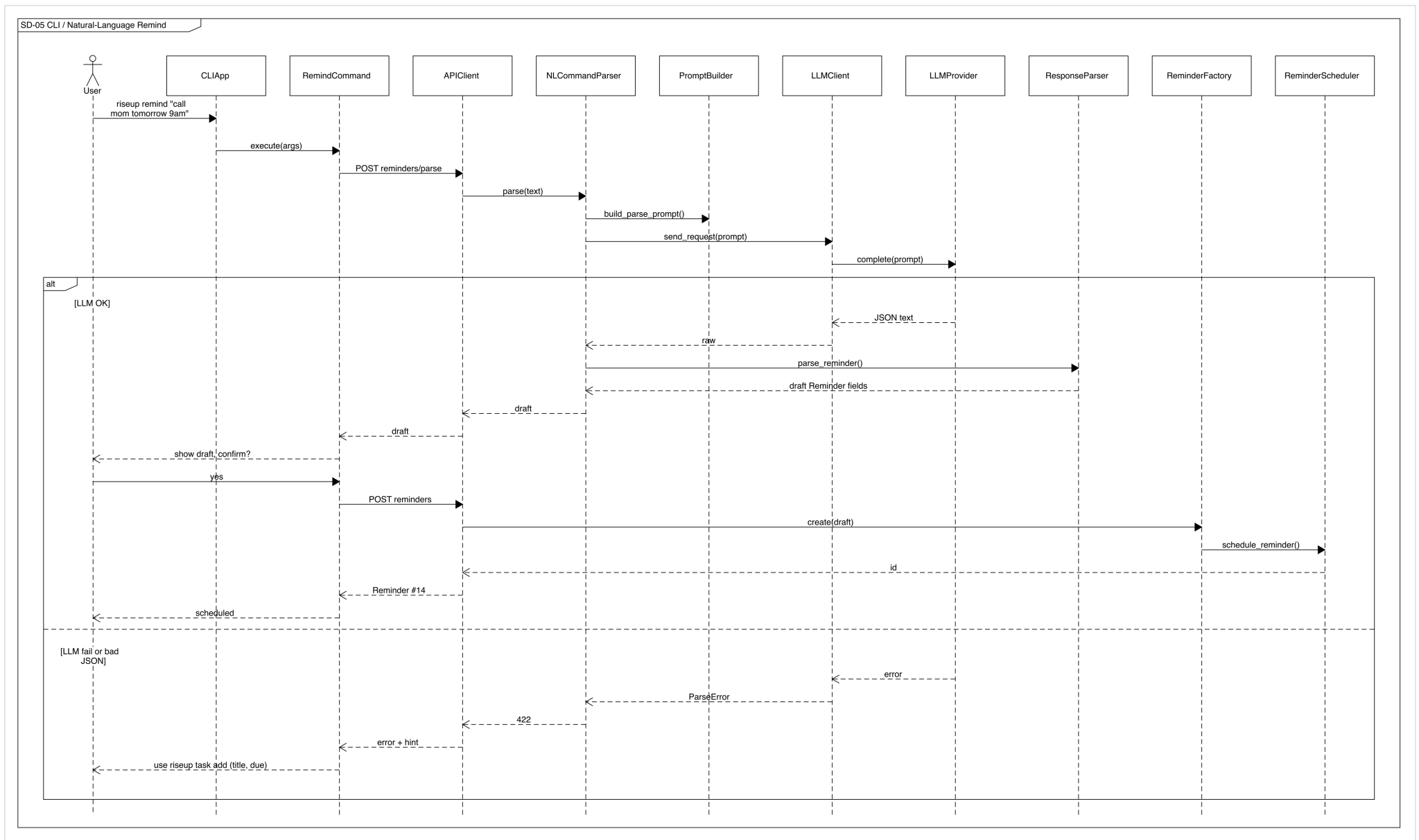


Figure 8: SD-05 — Sequence Diagram — CLI / Natural-Language Remind

Walkthrough

Participants: User, CLIApp, RemindCommand, APIClient, NLCommandParser, PromptBuilder, LLMClient, LLMPProvider, ResponseParser, ReminderFactory, ReminderScheduler.

Main flow.

1. The user runs `riseup remind "call mom tomorrow 9am"`. `CLIApp` calls `execute(args)` on `RemindCommand`.
2. `RemindCommand` sends `POST /reminders/parse` via `APIClient`. The server calls `NLCommandParser.parse(text)`, which calls `build_parse_prompt()` on `PromptBuilder` and then `send_request(prompt)` on `LLMClient`, which calls `LLMPProvider.complete(prompt)`.

alt [LLM OK]. The LLM returns JSON text. `NLCommandParser` passes the raw text to `ResponseParser.parse_reminder()`, which returns the draft reminder fields. The draft goes back through `APIClient` to `RemindCommand`, and the user is asked *"show draft, confirm?"*. When the user answers **yes**, `RemindCommand` sends `POST /reminders`. `ReminderFactory.create(draft)` runs, followed by `ReminderScheduler.schedule_reminder()`. The new id returns and the user sees *scheduled*.

alt [LLM fail / bad JSON]. The LLM call fails with an error and `NLCommandParser` raises `ParseError`. The API returns **422** with an error and a hint, and the user is told to use `riseup task add --title ... --due ...` instead.

Requirements realised: F07 (natural-language reminder creation) for UC-04 via the CLI. The GUI quick-add uses the same backend path. **Patterns:** Command (`RemindCommand`), Factory Method (`ReminderFactory`), Strategy (`LLMPProvider`) and Adapter (`LLMClient`). The user must confirm before anything is saved.

7.9 SD-06 — Sequence Diagram — AI Recommendations

- **File:** `uml-diagrams/SequenceDiagramRecommendations.png`
- **Purpose:** `RecommendationService` asks the agent/LLM for personalised alarm and escalation recommendations, parses them and lets the user apply them to `UserPreferences` .
- **Covers:** F13
- **Related:** UC-07

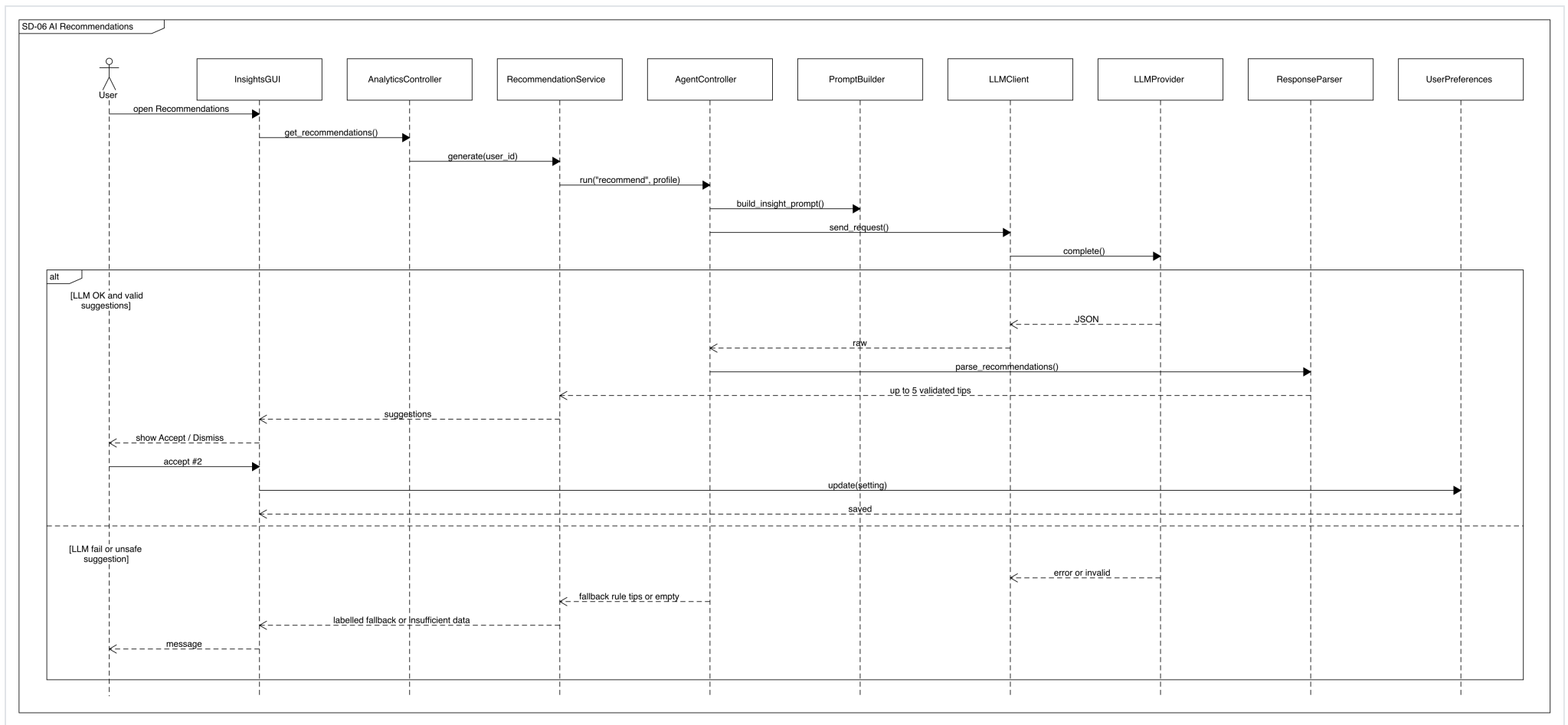


Figure 9: SD-06 — Sequence Diagram — AI Recommendations

Walkthrough

Participants: User, InsightsGUI, AnalyticsController, RecommendationService, AgentController, PromptBuilder, LLMClient, LLMPProvider, ResponseParser, UserPreferences.

Main flow.

1. The user opens *Recommendations*. `InsightsGUI` calls `get_recommendations()` on `AnalyticsController`, which calls `generate(user_id)` on `RecommendationService`.
2. `RecommendationService` calls `run("recommend", profile)` on `AgentController`. `AgentController` calls `build_insight_prompt()` on `PromptBuilder` and `send_request()` on `LLMClient`, which calls `LLMPProvider.complete()`.

alt [LLM OK + valid suggestions]. The JSON returns to `AgentController`, which calls `ResponseParser.parse_recommendations()`. Up to **5 validated tips** go back to `RecommendationService` and then to the GUI, which shows **Accept / Dismiss**. When the user accepts, `UserPreferences.update(setting)` runs and returns *saved*.

alt [LLM fail or unsafe suggestion]. The LLM returns an error or invalid output. `RecommendationService` returns fallback rule-based tips or an empty list, and the GUI shows a *labelled fallback / insufficient data* message.

Requirements realised: F13 (AI personalised recommendations) for UC-07. Each suggestion is validated against the settings schema, and unsafe suggestions (for example, one that disables all escalation) are dropped. Applying a suggestion is deterministic.

7.10 SD-07 — Sequence Diagram — History & Decision Explanation

- **File:** `uml-diagrams/SequenceDiagramHistoryExplain.png`
- **Purpose:** Viewing the alarm / escalation history and asking the system to explain why a decision was made.
- **Covers:** F14
- **Related:** UC-09

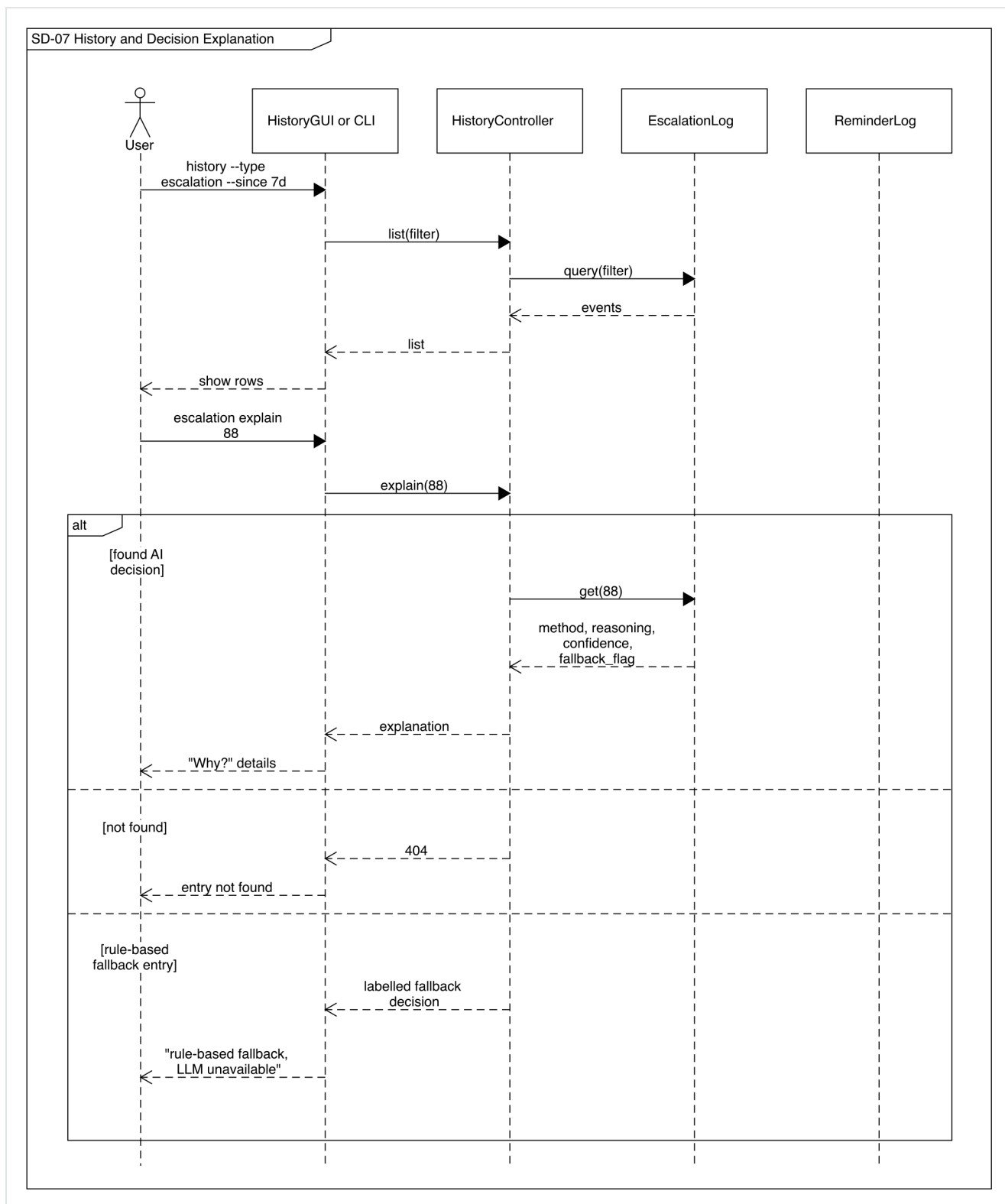


Figure 10: SD-07 — Sequence Diagram — History & Decision Explanation

Walkthrough

Participants: User, HistoryGUI_or_CLI, HistoryController, EscalationLog, ReminderLog.

Main flow.

1. The user runs `history --type escalation --since 7d`. The GUI or CLI calls `list(filter)` on `HistoryController`, which calls `query(filter)` on `EscalationLog` and gets back the matching events. The rows are shown to the user.
2. The user runs `escalation explain 88`, and `HistoryController.explain(88)` is called.

alt [found AI decision]. `HistoryController` calls `get(88)` on `EscalationLog` and receives the method, reasoning, confidence and fallback flag. The user sees the "Why?" details.

alt [not found]. `HistoryController` returns **404** and the user sees *entry not found*.

alt [rule-based fallback entry]. The decision is returned labelled as a fallback, and the user sees *"rule-based fallback, LLM unavailable"*.

Requirements realised: F14 (history and decision explanation) for UC-09. This meets the constraint that AI decisions must be explainable. Reminder history is read from `ReminderLog` in the same way.

Consolidated Design Analysis

Chapter overview. This chapter pulls together information that is spread across Chapters 1 – 7, so that each requirement can be traced to its design elements in one place. It contains no new requirements. Every entry comes from the feature specification, use cases, design decisions, patterns, traceability table or diagrams.

8.1 Full traceability matrix

Feature	Type	Use case(s)	Sequence	Pattern(s)	Design decisions	CLI command(s)	Primary fall-back / safeguard
F01 Alarm Management	Deterministic	UC-01	SD-01	—	AD-01, DP-01	<code>riseup alarm add/list/edit/delete/toggle</code>	20-alarm limit; DB failure → retry message, nothing scheduled
F02 Verification Challenge	Deterministic	UC-02	SD-01	Strategy, Factory Method	AD-02, AI-02	<code>riseup alarm verify <id> --answer N</code>	Camera unavailable → MathTask ; no response 2 min → F04
F03 Post-Wake Check-in	Deterministic	UC-02	SD-01	Observer, Adapter	UX-02	<code>riseup checkin confirm</code>	Push fails → SMS; timeout → F04
F04 AI Escalation Decision	AI / Hybrid	UC-03	SD-02	Strategy, Adapter	AD-03, AD-04, AD-05, AI-01, AI-03, UX-01	<code>riseup escalation explain <id></code>	LLM error / bad JSON / disabled method → fallback ladder
F05 Escalation Execution	Deterministic	UC-03	SD-02	Adapter, Observer	TR-02, UX-01	<code>riseup escalation status <id></code>	Channel fails → next channel; whole chain fails → "Alarm unanswered"
F06 Emergency Contact & Prefs	Deterministic	UC-08	prefs save (SD-01 style)	—	Constraints §3	<code>riseup prefs show/set , riseup contact add/remove/test</code>	No consent → contact excluded; all channels off → warning
F07 NL Reminder Creation	Hybrid	UC-04	SD-05	Command, Factory Method, Strategy	AI-01, AI-03	<code>riseup remind "<sentence>"</code>	Ambiguous → clarifying question; LLM down → manual form

Feature	Type	Use case(s)	Sequence	Pattern(s)	Design decisions	CLI command(s)	Primary fall-back / safeguard
F08 Reminder Management	Deterministic	UC-04, UC-06	SD-03	Factory Method	DP-01	riseup task add/list/edit/done/snooze/delete	Past time → offer tomorrow; DB failure → retry
F09 AI Reminder Prioritization	AI / Hybrid	UC-05	SD-03	Strategy	AI-02, AI-03	riseup task priority-preview	Invalid ranking → sort by priority → category → due time
F10 Reminder Escalation Levels	Deterministic	UC-05	SD-03	Observer	UX-03	riseup task done <id>	Completed early → chain cancelled; offline → queued
F11 Behavioral Learning	Deterministic	UC-03, UC-05, UC-07	SD-04	—	DP-03	riseup profile show/recompute	Too little data → defaults; job fails → previous profile kept
F12 Analytics Dashboard	Deterministic	UC-07	SD-04 (data path)	Strategy (formatter)	DP-02, DP-03	riseup stats --range 30d [--json]	No data → empty state; slow query → cached result
F13 AI Recommendations	AI	UC-07	SD-06	Strategy, Adapter	AD-05, AI-01, AI-03	riseup recommend [--apply n]	LLM down → rule-based tips; unsafe suggestion dropped
F14 History & Explain	Hybrid	UC-09	SD-07	—	AI-03 (stored reasoning)	riseup history, riseup escalation explain <id>	Not found → 404; fallback decisions labelled

8.2 Use-case coverage

Use case	Name	Primary actor	Features	Sequence diagram(s)	External actors
UC-01	Configure and Set an Alarm	User	F01	SD-01	—
UC-02	Alarm Verification & Check-in	System / User	F02, F03	SD-01	Camera, Firebase
UC-03	AI Escalation	System	F04, F05	SD-02	LLM, Twilio, Alexa
UC-04	Create a Task Reminder	User	F07, F08	SD-03, SD-05	LLM
UC-05	Send Prioritized Task Reminders	System	F09, F10	SD-03	LLM, Firebase
UC-06	Manage Existing Reminders	User	F08	SD-03	—
UC-07	View Analytics & Recommendations	User	F11, F12, F13	SD-04, SD-06	LLM

Use case	Name	Primary actor	Features	Sequence diagram(s)	External actors
UC-08	Manage Escalation Prefs & Emergency Contact	User	F06	prefs save	Twilio, emergency contact
UC-09	View History & Decision Explanation	User	F14	SD-07	—

Every feature F01 – F14 is covered by at least one use case, and every use case is covered by at least one feature and one sequence-diagram path.

8.3 Pattern coverage

Pattern	Problem solved	Key classes	Features supported
Strategy	Interchangeable verification tasks, LLM providers and CLI output formats	VerificationTask , LLMProvider , OutputFormatter and their concrete classes	F02, F04, F07, F09, F12, F13
Adapter	Hide vendor SDKs (Twilio, Alexa, Firebase, Anthropic / OpenAI) behind stable interfaces	Tool , TwilioClient , AlexaClient , FirebaseClient , LLMClient , ToolManager	F03, F04, F05, F07, F13
Command	Many CLI actions with uniform parsing, exit codes and testability	Command , concrete commands, CommandRegistry , CLIApp , APIClient	CLI access to all features (F01 – F14), notably F07
Observer	Several consumers react to status changes without coupling	Alarm / Reminder / CheckIn (subjects), ReminderObserver , DashboardNotifier , ReminderLogger , EscalationEngine (listener)	F03, F05, F08, F10
Factory Method	Centralised construction rules, defaults and fallbacks	VerificationTaskFactory , ReminderFactory	F02, F07, F08

8.4 Class catalogue

Class	Layer	Responsibility	Features
CLIApp	CLI	Entry point; parses argv, selects a command, sets exit code	All (CLI)
CommandRegistry	CLI	Maps command names to Command objects	All (CLI)
Command «interface»	CLI	execute(args) -> Result	All (CLI)
AddAlarmCommand , RemindCommand , StatsCommand , ListTasksCommand , ...	CLI	One concrete command per CLI action	F01, F07, F12, F08 ...
APIClient	CLI	HTTP requests to the backend; auth and error handling	All (CLI)
OutputFormatter «interface», TableFormatter , JsonFormatter	CLI	Format results as a table or JSON (--json)	F12 and all CLI output
AlarmController	Controller	Alarm CRUD entry point	F01
ReminderController	Controller	Reminder CRUD entry point	F08
PreferencesController	Controller	Update escalation preferences and contact	F06

Class	Layer	Responsibility	Features
AnalyticsController	Controller	get_stats(range) ; entry point for recommendations	F12, F13
HistoryController	Controller	list(filter) , explain(id)	F14
AgentController	Agent	Observe → decide → act → re-observe loop (run(goal, ctx))	F04, F05, F07, F13
EscalationEngine	Agent / service	Orchestrates escalation: context, LLM decision, validation, fallback_ladder()	F03, F04, F05
TaskPrioritizer	Agent / service	Ranks simultaneous reminders; deterministic fallback sort	F09
RecommendationService	Agent / service	Generates and validates personalised suggestions	F13
NLCommandParser	AI	Parses natural-language reminder text into fields	F07
PromptBuilder	AI	Builds structured prompts (escalation, parse, insight)	F04, F07, F13
LLMClient	AI	Provider selection, retries, timeouts (send_request())	F04, F07, F09, F13
LLMProvider «interface», ClaudeProvider , OpenAIProvider	AI	complete(prompt) -> str vendor call	F04, F07, F09, F13
ResponseParser	AI	Validates JSON (parse_decision , parse_reminder , parse_ranking , parse_recommendations)	F04, F07, F09, F13
ToolManager	Tools	Executes escalations and notifications through Tool adapters	F03, F05, F06, F10
Tool «interface»	Tools	execute(payload) -> ToolResult	F05
TwilioClient	Tools	Phone call and SMS adapter	F05, F06
AlexaClient	Tools	Alexa alert adapter	F05
FirebaseClient	Tools	Push notification adapter	F03, F10
User	Domain	Account; owns alarms, reminders, profile, preferences	All
UserProfile	Domain	Aggregated metrics; agent long-term memory	F04, F09, F11, F12, F13
UserPreferences	Domain	Enabled channels, quiet hours, settings	F01, F06, F13
EmergencyContact	Domain	Contact details and consent (request_consent())	F05, F06
Alarm	Domain	Alarm settings and lifecycle (trigger , stop , validate)	F01, F02
CheckIn	Domain	Post-wake check-in (schedule , record_response , timeout)	F03
Reminder	Domain	Reminder data and status (mark_complete , mark_missed)	F07 – F10
VerificationTask «abstract»	Verification	validate() , get_task_prompt()	F02
ObjectDetectionTask , MathTask , TriviaTask , CustomTask	Verification	Concrete challenge types	F02

Class	Layer	Responsibility	Features
VerificationTaskFactory	Verification	Creates the task for the user's preferences, with fallbacks	F02
ReminderFactory	Reminders	Creates Reminder from form data or parsed fields	F07, F08
ReminderScheduler	Scheduling	Schedules reminders; sends primary / escalation / final levels	F07 – F10
AlarmScheduler	Scheduling	Schedules and triggers alarms	F01, F02
ReminderObserver «interface», DashboardNotifier, ReminderLogger	Observer	React to status changes (UI and logs)	F10
LearningEngine	Learning	Nightly analyze_behavior() → profile update	F11
AnalyticsEngine	Learning	compute_metrics(), query()	F11, F12
AlarmRepository, ReminderRepository	Persistence	Save and load alarms and reminders	F01, F08
EscalationLog	Persistence	Escalation history: method, outcome, reasoning, confidence, fallback flag	F04, F05, F11, F14
ReminderLog	Persistence	Reminder history	F10, F11, F14

8.5 Design decision index

ID	Decision	Chosen option	Alternatives rejected
AD-01	System architecture	Layered client-server (React + FastAPI, REST + WebSocket)	Monolith; microservices
AD-02	Verification task design	Abstract VerificationTask + concrete strategies	if/else on a type string; single class with a mode flag
AD-03	Escalation logic location	Dedicated EscalationEngine orchestrator	Inline in Alarm; inside LLMClient
AD-04	LLM integration	LLMClient + LLMPProvider strategies; decisions stay in the domain/agent layer	Direct SDK calls everywhere; LLMClient.decide_escalation()
AD-05	Agent structure	AgentController multi-step loop with memory	One-shot "prompt → text" chatbot
AI-01	LLM providers	Claude claude-sonnet-4-5 + OpenAI gpt-4o behind LLMPProvider	Rule-based heuristics; custom ML model; random selection
AI-02	Scope of AI	AI decides; deterministic code executes	LLM for everything, including verification
AI-03	Prompt design	Structured prompts with a JSON output schema and constraints	Unstructured prompts
DP-01	Persistent storage	PostgreSQL (ACID, relational, queryable)	SQLite; MongoDB
DP-02	Caching / real-time	Redis (cache, sessions, pub/sub, Celery queue)	Database-only caching
DP-03	Metrics storage	Pre-aggregated metrics in UserProfile, updated nightly	Compute from raw logs every time
TR-01	Test strategy	pytest unit tests + KUMA behavioural tests	Unit tests only; behavioural tests only

ID	Decision	Chosen option	Alternatives rejected
TR-02	Error handling	Error handling and fallbacks for every external call	Fail hard
UX-01	Escalation order	AI decision first; deterministic ladder as fallback only	Fixed ladder only; AI only with no fallback
UX-02	Dismissal	Delayed check-in 5 – 10 min after verification	Immediate dismissal
UX-03	Reminder delivery	Adaptive levels (primary, +5 min, +15 min) that stop when the task is done	Single reminder; fixed repeats

8.6 Error-handling and fallback summary

Failure	System behaviour	Feature
LLM timeout or API error (escalation)	Deterministic fallback ladder: call → SMS → Alexa → emergency contact	F04
LLM returns malformed JSON	One retry, then fallback; decision logged with <code>fallback_used = true</code>	F04, F07
LLM picks a disabled method	Decision rejected; fallback used	F04
LLM drops or invents a reminder in a ranking	Ranking rejected; sort by priority → category → due time	F09
LLM unavailable for NL parsing	Manual form opened, pre-filled with the text (CLI: 422 + hint)	F07
LLM unavailable for recommendations	Rule-based tips only	F13
Unsafe recommendation	Suggestion dropped	F13
Camera unavailable / blurry photo	Fall back to MathTask / retry	F02
Wrong verification answer	Retry; alarm keeps ringing	F02
No response to alarm for 2 min	Hand over to escalation	F02 → F04
Push notification fails	Fall back to SMS	F03
Check-in not confirmed in 5 min	Escalation triggered	F03 → F04
Phone call unreachable	Wait 30 s, then SMS	F05
No Alexa device	Channel skipped	F05
Emergency contact unreachable	3 attempts, then log failure and notify the user	F05
Entire escalation chain fails	Dashboard shows "Alarm unanswered"	F05
All channels disabled	Warning at configuration; "missed alarm" logged	F04, F06
Emergency contact has not consented	Contact excluded from escalation	F06
Database failure on save	Retry message; nothing scheduled	F01, F08
Device offline for a reminder	Notification queued	F10
Learning job fails	Previous profile kept; error logged	F11
Insufficient data	Defaults kept; "insufficient data" shown	F11, F13
Slow analytics query	Cached result served	F12
History entry not found	404 message	F14

8.7 Risk register

These risks come from the constraints, assumptions and fallbacks documented in Chapters 1, 2 and 4.

#	Risk	Impact	Mitigation in the design
R1	LLM outage or high latency	Missed escalation or reminder ordering	AI never blocks execution; fallback ladder (UX-01), deterministic sort (F09), manual form (F07); TR-02
R2	LLM hallucination or invalid action	Wrong or impossible channel chosen	JSON schema and constraints (AI-03); allow-list of enabled methods; permutation check for rankings; settings-schema check for recommendations
R3	Third-party channel failure (Twilio, Alexa, Firebase)	User not reached	Multi-channel fallback chain (F05); adapters isolate vendors (Adapter pattern)
R4	Privacy or consent violation	Harm to the user or the contact	Emergency contact only after explicit consent; quiet hours; minimal personal data (F06, Constraints)
R5	Cold start: no history for a new user	Poor personalisation	Defaults and "insufficient data" state (F11, F13); fallback ladder still works
R6	User verifies and falls back asleep	Alarm goal defeated	Post-wake check-in (UX-02, F03)
R7	Notification fatigue	Users disable reminders	Escalation stops on completion (UX-03); repeatedly dismissed suggestion types are suppressed (F13)
R8	Unexplainable AI behaviour	Loss of trust	Reasoning, confidence and fallback flag stored for every decision (F14)
R9	Vendor lock-in	Costly provider changes	LLMProvider and Tool abstractions (AD-04, Adapter, Strategy)
R10	Growth beyond a single server	Missed triggers at scale	Redis caching, Celery jobs; future microservices and distributed scheduling (Chapter 4, Future Improvements)

8.8 Test strategy overview

Deterministic components: unit tests (pytest). These are fast, cheap and expected to pass every time (TR-01). The table lists example targets taken from the specified behaviour.

Component	Example assertions
Alarm / AlarmScheduler	Valid time accepted, invalid format rejected; 21st alarm rejected; job trigger time matches
MathTask , TriviaTask , CustomTask	Correct answer passes, wrong answer fails
VerificationTaskFactory	Camera unavailable → returns MathTask
CheckIn	Confirmation within 5 min → confirmed; timeout → escalation event emitted
EscalationEngine.fallback_ladder()	Order call → SMS → Alexa → emergency contact; skips disabled channels and non-consenting contacts
ToolManager (adapters mocked)	Channel failure → next channel tried; outcome logged
TaskPrioritizer fallback sort	Priority → category → due time ordering
ReminderScheduler	Primary / +5 min / +15 min sequence; cancelled when completed
LearningEngine / AnalyticsEngine	Success rates and completion rates computed correctly; corrupted rows skipped

Component	Example assertions
ResponseParser	Rejects malformed JSON, unknown methods and non-permutation rankings
CLI commands	Exit codes 0 / 1 / 2; <code>--json</code> output is valid JSON

AI / agent components: behavioural tests (KUMA). These are specified in TR-01:

ID	Scenario	Pass criterion
BR-01	Only SMS and Alexa enabled	Chosen method is in the enabled set; invalid picks rejected
BR-02	LLM returns malformed JSON	One retry, then fallback ladder; <code>fallback_used = true</code> logged
BR-03	Profile shows call 95 %, SMS 40 %	Agent prefers call when it is enabled

Candidate additional behavioural tests for Stage 3. These are suggested from the error cases in the feature specification:

ID	Scenario	Expected behaviour
BR-04	Several reminders due; LLM omits one	Ranking rejected; deterministic sort used (F09)
BR-05	Ambiguous NL input ("remind me to call mom")	Agent asks a clarifying question about the time (F07)
BR-06	Recommendation would disable all escalation	Suggestion dropped (F13)
BR-07	Emergency contact has not consented	Contact never chosen, even by the fallback ladder (F06)
BR-08	Call fails during escalation	Agent re-decides and uses the next channel (F05, SD-02)

8.9 Success-criteria checklist

Success criterion (Chapter 1 §7)	Where it is satisfied
Alarm cannot be dismissed without verification; check-in verifies wakefulness	F02, F03, UC-02, SD-01, UX-02
Escalation is agent-driven with deterministic fallback	F04, F05, UC-03, SD-02, AD-05, UX-01
CLI and GUI both reach the major features	Chapter 2 §3 (CLI command table), SD-05, SD-07
Learning updates profiles used by later decisions	F11, SD-04, DP-03
Decisions are explainable in history	F14, UC-09, SD-07
Deterministic components are unit-testable; agent behaviour is testable later with KUMA	TR-01, §8.8

Glossary

Term	Definition
Agent	Software that observes its context, decides on an action, acts through tools and observes the result in a loop. In RiseUp this is <code>AgentController</code>
Agent memory	Data the agent uses to decide. Short-term: the current alarm or reminder and the last tool result. Long-term: <code>UserProfile</code> and <code>EscalationLog</code>
Alarm	A scheduled wake-up event with sound, vibration, repeat pattern, snooze setting and verification task type
APScheduler	Python scheduling library used to trigger alarms and reminders
Celery	Background job queue (on Redis) for escalation and learning jobs
Check-in	A notification 5 – 10 min after successful verification asking "Are you still awake?"
CLI	Command-line interface (<code>riseup ...</code>), a thin client over the REST API
Confidence	A 0.0 – 1.0 value returned by the LLM with each decision
Deterministic	Behaviour that gives the same output for the same input, with no LLM
Emergency contact	A person contacted only as a last resort, and only after explicit consent
Escalation	Contacting the user through stronger channels (call, SMS, Alexa, emergency contact) when an alarm or check-in is ignored
Fallback ladder	Fixed deterministic escalation order used when the LLM fails (UX-01)
FastAPI	Python web framework used for the backend
Firebase (FCM)	Firebase Cloud Messaging, the push notification service
GUI	Graphical user interface: the React web app
Hybrid feature	A feature where AI decides or extracts and deterministic code validates and executes
KUMA	Framework used for behavioural tests of AI / agent components
LLM	Large language model (Claude <code>claude-sonnet-4-5</code> , OpenAI <code>gpt-4o</code>)
NL	Natural language, for example "Remind me to call mom tomorrow at 9am"
Observer	Pattern where subjects notify subscribed observers of status changes
PostgreSQL	Relational database used for persistent storage
Quiet hours	Time window in which escalation calls and SMS are not sent
Redis	In-memory store used for caching, sessions, pub/sub and the Celery queue
Reminder escalation levels	Primary reminder, escalation at +5 min, final at +15 min, then marked missed
REST / WebSocket	Request/response HTTP API and a persistent channel for real-time updates
SD	Sequence diagram (SD-01 – SD-07)
Strategy / Adapter / Command / Factory Method	The design patterns described in Chapter 5
Tool	An adapter through which the agent acts on the world (Twilio, Alexa, Firebase)
Twilio	Telephony service for phone calls and SMS
UC	Use case (UC-01 – UC-09)
UserProfile	Pre-aggregated behavioural metrics per user (DP-03)
Verification challenge	A task the user must complete to dismiss an alarm: photo object detection, math, trivia or custom

Conclusion and Next Steps

Summary

Stage 1 produced a complete, consistent design for RiseUp:

- A clear **problem and scope**, with target users, constraints (consent, explainability, minimal data) and measurable **success criteria**.
- **14 features**, each specified with inputs, outputs, AI involvement, a workflow at class/method level and error cases. They are reachable from both a **web GUI** and a **CLI**.
- **9 use cases** with full scenarios, alternative flows and exception flows.
- A **layered architecture** with a real **AI agent loop** (observe → decide → act → re-observe), two interchangeable LLM providers, and a strict rule that **AI decides while deterministic code executes**.
- **16 justified design decisions** and **5 design patterns**, each tied to a concrete design problem.
- **10 UML diagrams** (architecture, class, use-case and 7 sequence diagrams) that match the feature workflows one to one, as shown in the [traceability matrix](#).
- Reliability through **fallbacks for every AI and external dependency**, and trust through **stored, explainable decisions**.

Next steps

Stage 2: AI-human collaborative implementation

- ☐ Backend development (FastAPI controllers, services, agent loop, schedulers)
- ☐ Frontend development (React dashboard: alarms, tasks, insights, history, preferences)
- ☐ CLI (`riseup` commands, formatters, API client)
- ☐ API integrations (Claude, OpenAI, Twilio, Alexa, Firebase, vision)
- ☐ Database schema (PostgreSQL) and Redis caching
- ☐ AI collaboration logs

Stage 3: Testing & validation

- ☐ Unit tests for deterministic components (pytest)
- ☐ Integration tests (with mocked adapters)
- ☐ Agent behavioural tests with KUMA (BR-01 – BR-03, plus candidates BR-04 – BR-08)
- ☐ Performance testing

Possible future enhancements (from Chapter 4): an ML escalation model once enough data exists, wearable integration, calendar integration, Alexa voice commands, microservices, and distributed scheduling.